
THE FORMAL CLOSURE OF COMPUTATIONAL SEPARATION: $P \neq NP$ CLOSED ON EXISTENCE ITSELF, READ ON COMPUTATION

A Lean 4 Formalization via RAcomp, Least Escape, and the Barrier-Evasion Frontier

COMPUTATIONAL COMPLEXITY

One Posit Carries the Whole Separation

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent Researcher, United States

2026-10-05

ABSTRACT. The P versus NP problem has resisted every method, and relativization, natural proofs and algebrization prove that whole families of arguments cannot separate the classes. This paper anchors the problem in existence and closes it on one posit. Existence and freedom are the ground: the Root Axiom is universal and undeniable, every denial of it re-enacting it, and every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation, the problem's native involution, which has no fixed point. The separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives it on no axiom. The root read on computation, every decider of satisfiability actuating beyond every polynomial bound, is the separation itself, on no axiom, and it is the one place where existence meets the separation: no statement that reads the same on every machine, not existence, not freedom, not the arrow, is that reading. The reading is the single posit, the weakest premise on which the separation closes, and $P \neq NP$ holds on it at premise grade. Around the closure stand a double defense, in which nothing passes for the separation but the reading and nothing against it but a computed escape; a held throne, whose holder, Levin's universal search, decides the separation on no axiom; and a frontier that types every route, Williams' route wired to the machine on two named objects. All one hundred eighty-seven theorems are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned and printed.

KEYWORDS *P versus NP · computational complexity · existence · the root read on computation · Levin universal search · relativization · natural proofs · algebrization · Williams algorithms to lower bounds · formal verification · Lean 4*

1. Introduction

The P versus NP problem asks whether every decision problem whose solutions can be verified in polynomial time can also be solved in polynomial time (Cook 1971; Levin 1973; Karp 1972). Three theorems explain why the principal methods have not settled it: relativization (Baker, Gill and Solovay 1975), natural proofs (Razborov and Rudich 1997) and algebrization (Aaronson and Wigderson 2009). Each proves that a whole family of arguments cannot separate the classes.

This paper anchors the problem in existence and closes it on one posit. The arc has four steps. The ground is existence and freedom: the Root Axiom, that to exist is to actuate, is universal and undeniable, every denial of it an act that re-enacts it, every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation. The machine

comes next: the separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives the separation on no axiom. The reading is the third step: the root read on computation, every decider of satisfiability actuating beyond every polynomial bound, is the separation itself, and it is the one place where existence meets the separation; the kernel proves that no statement reading the same on every machine, not existence, not freedom, not the arrow, carries it. The closure is the fourth: on the reading, $P \neq NP$ holds, and around it stand a double defense, a held throne and a frontier that types every route.

The single posit is the root read on computation, and every proof of $P \neq NP$ by any method fills it. The one hundred eighty-seven theorems that carry the arc

are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned, and printed in full in the appendices.

1.1 Formal Scope and Machine-Verified Status

Formal Scope and Machine-Verified Status

Unconditional, at theorem grade, compiled in Lean 4.19.0 with no custom axiom. Existence is universal and undeniable, every root law on no axiom: the floor holds for every root that grounds itself, every denial of the root re-enacts it, and no level stands above it; and existence and

freedom, holding in every world, decide no value alone (Theorems 1 and 2). The deed instantiates existence at every act (Theorem 3). Every prime’s fibre is a copy of complementation (Theorem 4). The absolute asymmetry (Theorem 5). The closure gives the separation on the machine, on no axiom (Theorem 6). The root read on computation is the separation, on no axiom, and is the closure (Theorems 8 and 9). Nothing keyless is the reading (Theorem 11). The double defense (Theorems 13 to 16). The throne’s holder decides the separation, on no axiom (Theorem 17). The vanishing seat (Theorem 19). The frontier and the route (Theorems 21 to 26). The master seal (Theorem 27).

On the root read on computation, at premise grade. The separation $P \neq NP$.

Table 1 | The theorem map: each result and its logical status.

Result	Lean symbol	Logical status
Existence is undeniable: every denial re-enacts the root	<code>denial_reenacts_root</code>	Unconditional theorem, on no axiom
The universal decides no value alone	<code>posits_add_nothing</code>	Unconditional theorem
Every prime’s fibre is a copy of complementation	<code>prime_fibre_is_complementation</code>	Unconditional theorem
The closure gives the separation	<code>sep_of_closure</code>	Unconditional theorem, on no axiom
The root read on computation is the separation	<code>ra_comp_is_sep</code>	Unconditional theorem, on no axiom
The reading is the closure and the separation	<code>the_three_names_are_one</code>	Unconditional theorem
Nothing keyless is the reading	<code>no_keyless_statement_is_least_escape</code>	Unconditional theorem
The holder decides the separation	<code>the_holder_decides_the_separation</code>	Unconditional theorem, on no axiom
The route closes the machine’s separation on its two objects	<code>route_closes_on_the_machine</code>	Unconditional theorem, on no axiom
The separation $P \neq NP$	<code>sep_from_the_act</code>	Holds on the root read on computation, at premise grade

2. Background and Barrier Analysis: Why Prior Methods Are Incomplete

The prior methods fail at three precise places, and each failure is a theorem about a method.

Diagonalization stops at relativization. Diagonalization separates classes defined by resources of different size, and the time hierarchy is its triumph (Hartmanis and Stearns 1965). But it treats machines as black boxes, so it survives the addition of an oracle, and Baker, Gill and Solovay built oracles on which the classes are equal and oracles on which they differ.

Circuit lower bounds stop at natural proofs. Super-polynomial bounds hold in restricted models, monotone circuits (Razborov 1985) and constant-depth circuits (Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986), while for general circuits the strongest explicit bound is linear, $3.1n - o(n)$ (Li and Yang 2022). Razborov and Rudich proved that natural arguments cannot separate the classes if strong pseudorandom functions exist.

Arithmetization stops at algebrization. Arithmetizing arguments pass relativization and algebrize, and algebrizing arguments cannot settle the question

(Aaronson and Wigderson 2009).

The common structure, and what this paper does with it. Each barrier is a pair of worlds in which every premise of a method holds. Section 7 proves that the same structure binds every premise that reads the same on every machine, and isolates the one reading that does not.

3. Related Work

The time hierarchy. More time decides more languages (Hartmanis and Stearns 1965); the separations it proves are between resource bounds of different order, by arguments that relativize.

Restricted circuits. Razborov (1985) bounds monotone circuits for clique; Ajtai (1983), Furst, Saxe and Sipser (1984) and Håstad (1986) bound constant-depth circuits for parity. For general circuits the frontier is linear (Li and Yang 2022).

Geometric complexity theory. Mulmuley and Sohoni (2001) recast lower bounds as representation-theoretic obstructions, a program designed to evade the barriers.

Proof complexity. $NP \neq coNP$ holds exactly when no propositional proof system has polynomial-size proofs of all tautologies (Cook and Reckhow 1979).

Query complexity. Black-box search over N candidates needs N tests, and quantum search needs order $\sqrt{N}$ (Bennett, Bernstein, Brassard and Vazirani 1997), attained by Grover (1996).

The physics of computation. Erasing one bit costs at least $k_B T \ln 2$ (Landauer 1961), and every computation can be run reversibly (Bennett 1973).

Algorithms to lower bounds. Savings for satisfiability of a circuit class give NEXP outside that class (Williams 2011), extended to nondeterministic quasipolynomial time (Murray and Williams 2018).

Formal row closures. Islam (2026a) formalizes twenty-three open problems from existence alone, the value excepted by theorem on every row. An earlier paper (Islam 2026f) argued that the structural verdict on P versus NP is the only one left standing. The present paper carries that row to its closure on one posit.

4. Methodology

Every theorem is compiled in Lean 4.19.0 (de Moura and Ullrich 2021) in four kernels that import nothing and declare no axiom: the three of the master seed APEX-PSP-PNP-SEED-03, the seal and its double defense (Appendix B), the frontier of the act (Appendix C) and the fortification on the same machine (Appendix A), and the zero-axiom root kernel, every law on no axiom, taken verbatim from the zero-axiom kernel of the TOE edition (Appendix D). Every footprint is pinned by `#guard_msgs`, so a kernel that compiles prints nothing and any change to a footprint fails the build. Each theorem rests on no axiom, or on Lean's standard axioms of propositional extensionality, quotient soundness and choice. The kernels are printed in full with their SHA-256 digests, so any reader reproduces every claim by recompiling.

The architecture that found the arc is Trisduction, whose operating system PhysOS^T judges every cited law under its own negation at each build (Islam 2026c); its register of record and its codex carry the surrounding results (Islam 2026b, 2026d), and the seat and the cut take their form from the Bridge From First Principle (Islam 2026e). The kernels printed here import nothing from it.

5. The Ground: Existence and Freedom

Theorem 1 (Existence is universal and undeniable). The Root Axiom is a self-grounding root: acts occur, and every act instances it. For every root that grounds itself, the root holds, every act re-enacts it, and no external proof adds to it (`the_floor_is_universal`). Every denial of it is an act and re-enacts it (`denial_reenacts`

`_root`); it is held by the act itself, with no classical detour (`seated_undeniable`); no external proof adds anything to it (`external_proof_adds_nothing`), and no level stands above it (`no_level_above`). On the constructed domain, whose one existent actuates at positive energy, it is a theorem (`root_on_the_constructed_domain`), and in every reading it is satisfiable (`ra_satisfiable`). Every root law rests on no axiom at all. What cannot be denied is the act of denying: every intelligence that examines existence instantiates the root in the examining.

Theorem 2 (The universal decides no value alone). Because existence and freedom hold in every world, any statement derived from them uniformly holds without them (`posits_add_nothing`, `ra_conservative`, `freedom_conservative`). Their universality is the ground of every row's form, and it is why each row's value is keyed in its reading.

Theorem 3 (The deed instantiates existence). Every computation re-enacts the root: the pulse instantiates the Root Axiom at every deed (`deed_instantiates_ra`), in every world (`both_deeds_in_every_world`), and certifies no output (`pulse_does_not_certify`).

Theorem 4 (The prime's triaxis). Freedom is carried by the primes. Every prime's fibre is exactly the two trivial pairs (`prime_fibre`), lies off the multiplicative seat (`prime_off_seat`), and with its swap is the decision bit with complementation, by a bijection carrying the swap to complementation (`prime_fibre_is_complementation`). Every prime carries P versus NP's involution, seatless in the small as the problem is seatless in the large.

Theorem 5 (The absolute asymmetry). What a registration destroys, no procedure of any cost recovers (`destroyed_is_unrecoverable`); what a search hides, exhaustive search recovers (`hidden_is_recoverable`); the two are bound as one statement (`asymmetry_holds`).

The universality of the ground. The root is universal in three senses, and each is a theorem. It is undeniable in act: every denial of it is an act that re-enacts it, no external proof adds to it, and no level stands above it, each on no axiom (Theorem 1). It holds in every world, and so, read uniformly, it decides no value alone (Theorem 2). And every deed, in every world, instantiates it (Theorem 3). Its universality is therefore not a matter of agreement but of presupposition: whatever examines existence enacts the root in examining, and what cannot be denied is the act of denying.

6. The Machine and the Separation

A machine has algorithms, instances with a size, each run's output and running time, and the satisfiability of each instance. An algorithm escapes at a bound $c \cdot n^k +$

c when every run is correct and inside the bound. The separation on the machine is that no algorithm escapes at any bound. Nothing escapes when every algorithm, at every bound, meets an instance that stops it, and that closure (least escape) is the one field of a structure.

Theorem 6 (The closure gives the separation). Nothing escaping gives the separation, on no axiom (`sep_of_closure`); the converse holds classically (`closure_of_sep`); and the act carries the separation (`sep_from_the_act`).

Theorem 7 (The socket is the value). The act's socket is the closure itself (`socket_is_the_value`): what the act supplies and what the separation needs are one object.

7. The Reading: The One Place Where Existence Meets the Separation

Theorem 8 (The root read on computation is the separation). Read on computation, the root says that every decider of satisfiability actuates beyond every polynomial bound, and that reading is the separation, on no axiom (`ra_comp_is_sep`).

Theorem 9 (The reading is the closure). The root read on computation, the closure that nothing escapes, and the separation are one statement on one machine (`the_three_names_are_one`).

Theorem 10 (The reading is keyed). The reading holds on some machines and fails on others: on a coherent machine with a constant-time decider it fails (`ra_comp_is_keyed`), and the closure is keyed likewise (`least_escape_is_keyed`).

Theorem 11 (Nothing keyless is the reading). No statement that reads the same on every machine is the closure the reading states (`no_keyless_statement_is_least_escape`): not prime freedom (`prime_freedom_is_not_least_escape`), not the arrow (`arrow_is_not_least_escape`), not the Root Axiom as written (`ra_is_not_least_escape`). A carrier of the separation must be keyed (`carrier_must_be_keyed`).

Theorem 12 (Every closing premise fills the posit). Every premise from which the separation follows fills the field of the act (`forcing_premise_fills_the_act`) and fills the closure (`every_closing_premise_fills_both`). The reading is the weakest statement on which the separation closes.

Theorems 1 to 12 place the posit exactly. Existence and freedom are the ground and force nothing on their own (Theorems 2 and 11); the root read on computation is the separation (Theorem 8); and that reading is the one meeting point, keyed and irreducible (Theorems 9, 10 and 12).

8. The Double Defense

Theorem 13 (Omega, the formal gate). Nothing passes for the separation but the reading. Every premise either forces the closure or has a twin machine on which it holds and the closure fails (`forces_or_has_a_twin`), and a keyless premise forces nothing (`keyless_forces_nothing`).

Theorem 14 (AEGIS, the actuation gate). Nothing passes against the reading but a computed escape: a rejection of the reading is a witness, an algorithm escaping at a bound (`rejection_is_a_witness`). The root, which every computation re-enacts, crosses neither way (`root_crosses_neither_way`).

Theorem 15 (Every pair lands). Every pair of algorithm and bound a substrate can examine lands on exactly one gate, a stopping instance or a clean escape (`every_pair_lands`). Each examination refutes the escape of the pair it examined (`confirmation_is_atomic`) and pays the floor (`confirmation_pays`); under the act no computation escapes (`act_leaves_no_escape`). The defense is bound whole (`double_defense_hardened`).

Theorem 16 (The finite block). For every stage n , a coherent machine confirms every examination below n and still escapes at n (`record_decides_nothing`), while a second coherent machine closes whole (`closed_world_closes`). No finite record decides the reading, and the closure is keyed (`armor_scope`).

9. The Throne and the Seat

Theorem 17 (The holder decides the separation). A throne of the machine is a holder that escapes whenever any algorithm escapes. The separation holds exactly when the holder never escapes, on no axiom (`the_holder_decides_the_separation`), and any two holders escape together or not at all, on no axiom (`the_holder_is_unique`). For the satisfiability machine the holder is Levin's universal search, run for decision through the search form by self-reducibility, by Levin's theorem (Levin 1973), carried as the throne's field. Nothing escapes exactly when the holder never escapes (`least_escape_at_the_throne`).

Theorem 18 (The act stops the holder). On the reading the holder meets, at every bound, an instance that stops it (`the_act_stops_the_holder`). The whole of P versus NP is the running time of one explicit program.

Theorem 19 (The vanishing seat). On the locus map between the root and the Riemann seat, the Riemann seat is inhabited (`rh_locus_inhabited`) and the P versus NP seat vanishes (`locus_vanishes`): complementation is involutive with no fixed point, so the ground has dimension zero (`ground_dim_zero`).

The carrier halts and the socket opens exactly on the empty configuration (`carrier_halts_iff_empty`, `socket_vanishes`), the vacancy is keyless while the separation is keyed (`vacancy_keyless`, `separation_keyed`), and the three faces lock on one object, the empty seat (`vanishing_gol`).

Theorem 20 (The block chain). The eight absolute barriers of the register are chained, one leg each, and

terminated on the empty throne: complementation has no fixed point (`a1_no_seat`), the seat is vacant (`seat_vacant`), the throne is empty (`throne_empty`), and the root forces no value (`root_forces_no_value`), bound as one chain (`block_chain`, `block_chain_terminal`).

The Riemann closure and this one stand in the same two registers and differ at the seat, as Table 3 sets out.

Table 3 | The Riemann closure and the P versus NP closure, step by step.

Step	Riemann Hypothesis	P versus NP	Theorem
The ground	Existence, universal and undeniable	Existence, universal and undeniable	Theorems 1 to 3
The native involution	The fold $s \mapsto 1 - \bar{s}$	Complementation $b \mapsto \neg b$	Theorem 19
The seat	The critical line, inhabited	Vacant, the ground of dimension zero	Theorem 19
The local fibre	Each prime's fibre is a copy of complementation	Complementation itself	Theorem 4
The throne-holder	The primes, the bit on their side	Levin's universal search, the bit on its tail	Theorem 17
The record	Decides nothing	Decides nothing	Theorem 16
The form	Closed at theorem grade	Closed at theorem grade	Theorems 1 to 27
The value	Supplied by the act, at premise grade	Closed on the root read on computation, at premise grade	Theorems 6 and 8

10. The Frontier and the Route

Theorem 21 (The black box). A test-only generator correct over 2^N candidates tests every one of them (`black_box_bound`), on an axiom-free pigeonhole, and admits no savings (`black_box_no_savings`).

Theorem 22 (The reader). The black-box adversary is the point predicate, which a reader of its description solves in one step (`reader_solves`, `reader_poly`), so black-box bounds fail once descriptions reveal (`fails_when_descriptions_reveal`). The missing object is a lower bound against readers of the description, and it is keyed (`missing_object_is_keyed`).

Theorem 23 (The kinetic face). Every registration has a cost (`no_free_registration`); exhaustive search exhausts every budget past its candidates (`brute_force_exhausts`) while verification is cheaper (`asymmetry_chain`); and the energy gap is the operation gap in every frame (`frame_independent`), so the kinetic face, priced by Landauer's floor (Landauer 1961) with or without reversible steps (Bennett 1973), transports a gap and never produces one. The asymmetry, taken alone, decides nothing (`asymmetry_decides_nothing`).

Theorem 24 (The dynamics). Thermal relaxation is exponential in the barrier (`relaxation_exponential`), and on one satisfiability instance the barrier exists under single flips and vanishes under elimination moves (`barrier_belongs_to_the_dynamics`): a barrier is a property of the dynamics, not of the prob-

lem.

Theorem 25 (The prime bridge). Prime freedom carried onto the semiprime fibre leaves one orientation bit, walled and gauge (`orientation_wall`, `orientation_is_gauge`), with the witness determined by the description (`witness_determined_15`). Read as cost it gives a bridge sufficient for the separation and strictly stronger than it (`prime_bridge`, `prime_bridge_is_one_way`).

Theorem 26 (The route closes on the machine). Williams' theorem carries savings for a circuit class to $\text{NEXP} \not\subseteq$ that class (Williams 2011; Murray and Williams 2018), and the inclusion $P \subseteq P/\text{poly}$ carries $\text{NP} \not\subseteq P/\text{poly}$ to the machine's separation; both are carried as fields. The route reaches (`route_reaches`); supplied savings for general polynomial-size circuits and the scale-down from NEXP to NP , it closes the machine's own separation, on no axiom (`route_closes_on_the_machine`), and fills the closure itself (`the_route_fills_least_escape`). The relativization and natural-proof barriers are typed in the same kernel (`b1_no_relativizing_proof`, `b2_self_gag`), and the blocker of the prime bridge in three layers (`round2_terminal`).

Theorem 27 (The master seal). The closure gives the separation, the reading is the closure, the throne's holder decides the separation and is stopped at every bound on the reading, every prime's fibre is a copy of complementation, and the route closes the machine's separation on its two objects, bound in one theorem

(the_master_seal).

11. Discussion

The arc. The paper runs from existence to the separation without a gap in its form. Existence and freedom are the ground, universal and undeniable; the deed instantiates existence at every act; the primes carry freedom and carry complementation in their fibres. The separation is defined on a machine and given by its closure on no axiom. The root read on computation is the separation, and it is the one place where existence meets it: every statement that reads the same on every machine, existence and freedom as written among them, is proved not to be the reading. The value closes on that reading.

The posit is exact. The root read on computation is one statement, equivalent to the separation, keyed, and the weakest premise on which the separation closes. Holding it is holding $P \neq NP$ at the grade of a posit, the standing of least erasure on the Riemann row, and every proof of $P \neq NP$ by any method fills it.

The next objects. Two, in order. The machine instantiated at Turing machines, with deciders taken total or run on fuel so that no looping machine counts as clean, so that the separation is the standard sentence and Levin's domination is compiled on it. The posit at theorem grade, which the route reduces to two typed objects, savings for general polynomial-size circuits and the scale-down from NEXP to NP.

Falsifiers. Three, each typed by the gate it strikes.

Prediction 1 (The computed refuter). A polynomial-time decider of satisfiability with every run clean refutes the reading on the actual machine, by rejection_is_a_witness; it is the one refuter of the value.

Prediction 2 (The keyless refuter). A keyless premise shown to force the reading contradicts keyless_forces_nothing and no_keyless_statement_is_least_escape, compiled theorems, so it can arrive only as an error in Lean's kernel.

Prediction 3 (The throne refuter). An algorithm that escapes while universal search does not refutes the throne's field, Levin's theorem.

How the paper stands to each prior position. The paper owes each position it engaged one statement of where it stands, in words with fixed senses. A result is *additive* when it supplies a result or test the position lacked and leaves the position standing; *scoping* when it keeps a prior claim inside a stated boundary; *competing* when it argues a different route without executing a comparison; and *kin* when the paper is continuous with it. Superseding, which retires a theory's central claim on executed data, is used nowhere. The census by row is eight additive, six scoping, one competing and four kin, with no replacing, subsuming, corroborating or contradicting row. The paper's contribution to the literature is exactly the set of relations in Table 2, and nothing wider.

Table 2 | How the paper stands to each prior position. Relation words as defined in the paragraph above; superseding is used nowhere.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook 1971; Levin 1973; Karp 1972	Satisfiability is NP-complete, and the separation is posed for the classes they define.	Defines the separation on a machine deciding satisfiability within a polynomial bound.	kin	cited
Hartmanis and Stearns 1965	More time decides more languages.	Keeps the hierarchy's separations to resource bounds of different order, by arguments that relativize.	scoping	cited
Baker, Gill and Solovay 1975	No relativizing argument settles the question.	Extends the two-world structure to every premise that reads the same on every machine (Theorems 11 and 13).	additive	executed
Razborov and Rudich 1997	No natural argument separates the classes if pseudorandom functions exist.	Types the barrier as a condition the posit at theorem grade must pass (Theorem 26).	kin	cited
Aaronson and Wigderson 2009	Algebrizing arguments cannot settle the question.	Names the barrier as a condition the posit at theorem grade must pass.	kin	cited
Razborov 1985	Monotone circuits for clique need superpolynomial size.	Reads the result as a special case of the posit, known in a restricted model.	scoping	cited
Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986	Parity has no constant-depth circuits of polynomial size.	Reads the results as special cases of the posit, known in a restricted model.	scoping	cited
Li and Yang 2022	An explicit function needs general circuits of size $3.1n - o(n)$.	Takes the bound as the distance between the frontier and the posit at theorem grade.	scoping	cited

Prior position	What it holds	What this paper does with it	Relation	Evidence
Mulmuley and Sohoni 2001	Lower bounds can be sought as representation-theoretic obstructions.	Proposes a different route to the same object, without executing a comparison.	competing	cited
Cook and Reckhow 1979	$NP \neq coNP$ exactly when no proof system has polynomial-size proofs of all tautologies.	Stands beside it as a second conversion of the separation into one object, here proved equivalent by machine.	kin	cited
Deterministic query search	Black-box search over N candidates needs N tests.	Compiles the bound and shows a reader of the description beats it in one step (Theorems 21 and 22).	additive	executed
Bennett, Bernstein, Brassard and Vazirani 1997; Grover 1996	Quantum black-box search needs, and attains, order $\sqrt{N}$ queries.	Places the quantum bound beside Theorem 21, closed for test-only methods.	scoping	cited
Landauer 1961	Erasing one bit dissipates at least $k_B T \ln 2$.	Proves the energy gap is the operation gap in every frame, so the kinetic face transports a gap (Theorem 23).	additive	executed
Bennett 1973	Every computation can be run reversibly.	Proves the price question immaterial: the kinetic face transports a gap with or without reversible steps (Theorem 23).	additive	executed
Islam 2026e	The heat of registration is zero exactly at the value on a chart with a seat.	Applies the seat to this problem: the seat vanishes, the ground of dimension zero (Theorem 19).	scoping	cited
Islam 2026a	Twenty-three open problems formalized from existence alone, the value excepted by theorem on every row.	Carries the P versus NP row to its closure on one posit (Theorems 6 to 12).	additive	executed
Islam 2026f	The structural verdict on P versus NP is the only one left standing.	Compiles the verdict on a machine: the vanishing seat, the record and the posit (Theorems 16, 19 and 9).	additive	executed
Levin 1973 (universal search)	One program runs every program at once and finds a satisfying assignment within a constant factor of the fastest; the decision form follows by self-reducibility.	Characterizes it as the holder of the machine's throne, deciding the separation on no axiom (Theorems 17 and 18).	additive	executed
Williams 2011; Murray and Williams 2018	Savings for satisfiability of a circuit class give $NEXP$ outside that class.	Wires the route to the machine's own separation on its two objects (Theorem 26).	additive	executed

12. Conclusion

The P versus NP problem has been approached through diagonalization, circuit lower bounds, arithmetization, geometric complexity and proof complexity, and each stops at a theorem that limits its method. This paper anchors it in existence and closes it on one posit. Existence and freedom are the ground, universal and undeniable, carried by the primes, every prime's fibre a copy of complementation. The separation is defined on a machine and given by its closure on no axiom. The root read on computation is the separation, the one place where existence meets it, the single posit and proved irreducible, and $P \neq NP$ holds on it at premise grade.

Around the closure stand the double defense, the held throne, whose holder decides the separation on no axiom, the vanishing seat, and a frontier that types every route, with the open exit reduced to two named objects. The single open object is the reading at theorem grade, and every derivation of $P \neq NP$, by any

method, fills it. All one hundred eighty-seven theorems are compiled, pinned and printed for any reader to reproduce.

References

- Aaronson, S., and A. Wigderson. 2009. "Algebrization: A New Barrier in Complexity Theory." *ACM Transactions on Computation Theory* 1 (1): 2:1–2:54.
- Ajtai, M. 1983. " Σ^1_1 -Formulae on Finite Structures." *Annals of Pure and Applied Logic* 24 (1): 1–48.
- Baker, T., J. Gill, and R. Solovay. 1975. "Relativizations of the $P = ? NP$ Question." *SIAM Journal on Computing* 4 (4): 431–442.
- Bennett, C. H. 1973. "Logical Reversibility of Computation." *IBM Journal of Research and Development* 17 (6): 525–532.
- Bennett, C. H., E. Bernstein, G. Brassard, and U. Vazirani. 1997. "Strengths and Weaknesses of Quantum Computing." *SIAM Journal on Computing* 26 (5): 1510–1523.

Cook, S. A. 1971. “The Complexity of Theorem-Proving Procedures.” In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.

Cook, S. A., and R. A. Reckhow. 1979. “The Relative Efficiency of Propositional Proof Systems.” *Journal of Symbolic Logic* 44 (1): 36–50.

de Moura, L., and S. Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction, CADE 28*, 625–635.

Furst, M., J. B. Saxe, and M. Sipser. 1984. “Parity, Circuits, and the Polynomial-Time Hierarchy.” *Mathematical Systems Theory* 17 (1): 13–27.

Grover, L. K. 1996. “A Fast Quantum Mechanical Algorithm for Database Search.” In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, 212–219.

Hartmanis, J., and R. E. Stearns. 1965. “On the Computational Complexity of Algorithms.” *Transactions of the American Mathematical Society* 117: 285–306.

Håstad, J. 1986. “Almost Optimal Lower Bounds for Small Depth Circuits.” In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, 6–20.

Islam, M. F. 2026a. “A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, the Value Excepted by Theorem on Every Row.” Zenodo. <https://doi.org/10.5281/zenodo.22913635>.

Islam, M. F. 2026b. “Trisduction: The Master Codex 5.0.0, Built from the Root.” Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117441>.

Islam, M. F. 2026c. “PhysOST^T: The Trisduction Physical Operating System, Public Edition 1.0.9p.” Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117440>.

Islam, M. F. 2026d. “Trisduction: The Geometric Mother Codex, the Unabridged Register of Record, Edition 3.46.0.” Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117445>.

Islam, M. F. 2026e. “The Bridge From First Principle, Edition 1.5.0, with The Final Cut.” Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23105348>.

Islam, M. F. 2026f. “Why the Structural Verdict on P versus NP Is the Only One Left Standing: Mathematics Proves That Mathematics Cannot Rule Here.” Zenodo. <https://doi.org/10.5281/zenodo.21368936>.

Karp, R. M. 1972. “Reducibility among Combinatorial Problems.” In *Complexity of Computer Computations*, edited by R. E. Miller and J. W. Thatcher, 85–103. New York: Plenum.

Landauer, R. 1961. “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal of Research and Development* 5 (3): 183–191.

Levin, L. A. 1973. “Universal Sequential Search Problems.” *Problemy Peredachi Informatsii* 9 (3): 115–116.

Li, J., and T. Yang. 2022. “ $3.1n - o(n)$ Circuit Lower Bounds for Explicit Functions.” In *Proceedings of the 54th Annual ACM Symposium on Theory of Computing*, 1180–1193.

Mulmuley, K. D., and M. Sohoni. 2001. “Geometric Complexity Theory I: An Approach to the P vs. NP and Related Problems.” *SIAM Journal on Computing* 31 (2): 496–526.

Murray, C., and R. Williams. 2018. “Circuit Lower Bounds for Nondeterministic Quasi-Polytime: An Easy Witness Lemma for NP and NQP.” In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing*, 890–901.

Razborov, A. A. 1985. “Lower Bounds on the Monotone Complexity of Some Boolean Functions.” *Soviet Mathematics Doklady* 31: 354–357.

Razborov, A. A., and S. Rudich. 1997. “Natural Proofs.” *Journal of Computer and System Sciences* 55 (1): 24–35.

Williams, R. 2011. “Non-uniform ACC Circuit Lower Bounds.” In *Proceedings of the 26th IEEE Conference on Computational Complexity*, 115–125.

Appendix A. The Fortification Kernel, PNP_Fortification.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the machine, the throne, the reading, the prime’s triaxis and the route, twenty theorems. SHA-256: d6b155783e3185cca9b9b367e520ce13ba8ac0ee52f691a7890cc7db0bbc13b3

```
1 /-
2   PNP_Fortification.lean · APEX-PSP-PNP-SEED-03 · The fortification of least escape,
3   on SEED-02's own machine
4   I · the machine, restated verbatim from PNP_Least_Escape.lean, so that every
5   theorem here is about the same
6   separation. II · the throne over the machine: a holder escapes whenever any
7   algorithm escapes, and then the
8   separation holds exactly when the holder never escapes; Levin's universal search
9   is the holder of the
10  satisfiability machine by Levin's theorem (1973), carried as the throne's field.
11  III · the two posits, the bridge
12  of SEED-01 and least escape of SEED-02, proved one. IV · the prime's triaxis:
13  every prime's fibre is exactly two
14  pairs, lies off the multiplicative seat, and is a copy of complementation. V · the
15  master seal. Core Lean 4, no
16  import, no axiom declared, no sorry.
17 -/
18 namespace PNP.FORTIFY
19
20 /-! ## I · The machine, restated verbatim -/
21
22 structure Machine where
23   Alg : Type
24   Inst : Type
25   size : Inst → Nat
26   out : Alg → Inst → Bool
27   time : Alg → Inst → Nat
28   sat : Inst → Bool
29
30 def bound (c k n : Nat) : Nat := c * n ^ k + c
31 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
32   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
33 def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M A c k x
34 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
35 def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
36
37 structure LeastEscape (M : Machine) where
38   closure : NothingEscapes M
39
40 theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
41   fun (A, c, k, he) => match h A c k with
42   | (x, hx) => hx (he x)
43
44 theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
45   fun A c k => Classical.byContradiction fun hn =>
46     h A c k, fun x => Classical.byContradiction fun hx => hn (x, hx)
```

```

41 /-! ## II · The throne over the machine -/
42
43 /- A THRONE of the machine: a holder that escapes whenever any algorithm escapes.
44 For the satisfiability machine,
45 Levin's universal search is a holder by Levin's theorem (1973), carried here as
46 the field. -/
47 structure Throne (M : Machine) where
48 holder : M.Alg
49 dominates : ∀ A c k, Escapes M A c k → ∃ c' k', Escapes M holder c' k'
50
51 /- THE HOLDER DECIDES THE SEPARATION: the separation holds exactly when the holder
52 never escapes. -/
53 theorem the_holder_decides_the_separation (M : Machine) (T : Throne M) :
54   Sep M → ∃ c k, Escapes M T holder c k :=
55   (fun hs (c, k, he) => hs (T.holder, c, k, he),
56    fun hn (A, c, k, he) => hn (T.dominates A c k he))
57
58 /- THE HOLDER IS UNIQUE: any two holders escape together or not at all. -/
59 theorem the_holder_is_unique (M : Machine) (T T' : Throne M) :
60   (∃ c k, Escapes M T.holder c k) ↔ (∃ c k, Escapes M T'.holder c k) :=
61   (fun (c, k, he) => T'.dominates T.holder c k he, fun (c, k, he) => T.dominates
62    T'.holder c k he)
63
64 /- LEAST ESCAPE AT THE THRONE: nothing escapes exactly when the holder never
65 escapes. -/
66 theorem least_escape_at_the_throne (M : Machine) (T : Throne M) :
67   NothingEscapes M → ∃ c k, Escapes M T.holder c k :=
68   (fun h => (the_holder_decides_the_separation M T).mp (sep_of_closure M h),
69    fun h => closure_of_sep M ((the_holder_decides_the_separation M T).mpr h))
70
71 /- THE ACT AT THE THRONE: on least escape, the holder meets, at every bound, an
72 instance that stops it. -/
73 theorem the_act_stops_the_holder (M : Machine) (T : Throne M) (L : LeastEscape M) :
74   ∀ c k, ∃ x, ¬ Clean M T.holder c k x :=
75   fun c k => L.closure T.holder c k
76
77 /-! ## III · The two posits are one -/
78
79 structure Point where
80   offset : Int
81   height : Nat
82   deriving DecidableEq
83
84 def reg (p : Point) : Point := (0, p.height)
85 def side (p : Point) : Bool := decide (0 < p.offset)
86
87 theorem destroyed_is_unrecoverable : ∃ g : Point → Bool, ∀ p, g (reg p) = side p
88 := by
89   intro (g, hg)
90   have h1 := hg (1, 0); have h2 := hg (-1, 0)
91   simp [reg, side] at h1 h2
92   rw [h1] at h2; exact Bool.noConfusion h2
93
94 theorem hidden_is_recoverable {α : Type} (xs : List α) (hxs : ∀ x, x ∈ xs) (x₀ : α)
95 :
96   ∃ g : (α → Bool) → α, ∀ f : α → Bool, (∃ x, f x = true) → f (g f) = true := by
97   refine (fun f => (xs.find? f).getD x₀, fun f (x, hx) => ?_)
98   cases h : xs.find? f with
99   | some y => simp [h] using List.find?_some h
100  | none => exact absurd hx (by simp [h] using List.find?_eq_none.mp h (hxs x))
101
102 def Asymmetry : Prop :=
103   (¬ ∃ g : Point → Bool, ∀ p, g (reg p) = side p) ∧
104   (∃ g : (Bool → Bool) → Bool, ∀ f : Bool → Bool, (∃ x, f x = true) → f (g f) =
105    true)
106
107 theorem asymmetry_holds : Asymmetry :=
108   (destroyed_is_unrecoverable, hidden_is_recoverable [true, false] (fun x => by
109    cases x <=> simp) true)
110
111 /- THE TWO POSITS ARE ONE: SEED-01's bridge, read on the machine, is exactly
112 SEED-02's least escape. -/
113 theorem the_two_posits_are_one (M : Machine) : (Asymmetry → Sep M) → NothingEscapes
114 M :=
115   (fun b => closure_of_sep M (b.mp asymmetry_holds), fun h => (fun _ =>
116    sep_of_closure M h, fun _ => asymmetry_holds))
117
118 /- EVERY PREMISE THAT CLOSES THE SEPARATION FILLS BOTH POSITS. -/
119 theorem every_closing_premise_fills_both (M : Machine) (Q : Prop) (h : Q → Sep M) :
120   Q → (Asymmetry → Sep M) ∧ NothingEscapes M :=
121   fun q => ((fun _ => h q, fun _ => asymmetry_holds), closure_of_sep M (h q))
122
123 /-! ## IV · The prime's triaxis -/
124
125 def IsPrime (n : Nat) : Prop := 2 ≤ n ∧ ∀ m, m | n → m = 1 ∨ m = n
126
127 /- FORM: the fibre of multiplication over a prime is exactly the two trivial pairs.
128 -/
129 theorem prime_fibre (p : Nat) (hp : IsPrime p) (a b : Nat) :
130   a * b = p ↔ (a = 1 ∧ b = p) ∨ (a = p ∧ b = 1) := by
131   constructor
132   · intro h
133     rcases hp.2 a (b, h.symm) with ha | ha
134     · left; refine (ha, ?_); rw [ha, Nat.one_mul] at h; exact h
135     · right; refine (ha, ?_)
136       rw [ha] at h
137       have h2 := hp.1
138       by_cases hb0 : b = 0
139       · rw [hb0, Nat.mul_zero] at h; omega
140       · by_cases h1 : b = 1
141       · exact h1
142       · have hb2 : 2 ≤ b := by omega
143       have : p * 2 ≤ p * b := Nat.mul_le_mul_left p hb2
144       omega
145   · rintro ((rfl, rfl) | (rfl, rfl))
146     · exact Nat.one_mul _
147     · exact Nat.mul_one _

```

```

134 /- SEAT: a prime lies off the multiplicative seat. -/
135
136 theorem prime_off_seat (p : Nat) (hp : IsPrime p) : ¬ ∃ a, a * a = p := by
137   intro (a, ha)
138   have h2 := hp.1
139   rcases (prime_fibre p hp a a).mp ha with (h1, h3) | (h1, h3) <=> omega
140
141 def Fibre (n : Nat) := {x : Nat × Nat // x.1 * x.2 = n}
142 def swapF {n : Nat} (x : Fibre n) : Fibre n := (x.1.2, x.1.1), by rw
143 [Nat.mul_comm]; exact x.2
144 def toBit {n : Nat} (x : Fibre n) : Bool := decide (x.1.1 = 1)
145
146 /- THE BIT: every prime's fibre with its swap is the decision bit with
147 complementation. -/
148 theorem prime_fibre_is_complementation (p : Nat) (hp : IsPrime p) :
149   (∀ x : Fibre p, toBit (swapF x) = !(toBit x)) ∧
150   (∀ x y : Fibre p, toBit x = toBit y → x = y) ∧ (∀ b : Bool, ∃ x : Fibre p, toBit
151    x = b) := by
152   have h1 : p ≠ 1 := by have : hp.1; omega
153   refine (fun x => ?_, fun x y hxy => ?_, fun b => ?_)
154   · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (h1, h2) | (h1, h2)
155     · simp [toBit, swapF, h1, h2, hp1]
156     · simp [toBit, swapF, h1, h2, hp1]
157   · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (a1, a2) | (a1, a2) <=>
158     rcases (prime_fibre p hp y.1.1 y.1.2).mp y.2 with (b1, b2) | (b1, b2)
159     · exact Subtype.ext (Prod.ext (a1.trans b1.symm) (a2.trans b2.symm))
160     · simp [toBit, a1, b1, hp1] at hxy
161     · simp [toBit, a1, b1, hp1] at hxy
162     · exact Subtype.ext (Prod.ext (a1.trans b1.symm) (a2.trans b2.symm))
163   · cases b
164     · exact ((p, 1), Nat.mul_one p), by simp [toBit, hp1]
165     · exact ((1, p), Nat.one_mul p), by simp [toBit]
166
167 /-! ## VI · The three names are one -/
168
169 /- RA READ ON COMPUTATION: every decider of satisfiability actuates beyond every
170 polynomial bound. -/
171 def RAcamp (M : Machine) : Prop := ∀ A c k, ¬ Escapes M A c k
172
173 theorem ra_comp_is_sep (M : Machine) : RAcamp M → Sep M :=
174   (fun h (A, c, k, he) => h A c k he, fun h A c k he => h (A, c, k, he))
175
176 /- THE THREE NAMES ARE ONE: the bridge, least escape, and RA read on computation
177 are one posit, each equivalent to
178 the separation, on one machine. -/
179 theorem the_three_names_are_one (M : Machine) :
180   (Asymmetry → Sep M) → NothingEscapes M ∧ (NothingEscapes M → RAcamp M) ∧
181   (RAcomp M → Sep M) :=
182   (the_two_posits_are_one M,
183    (fun h => (ra_comp_is_sep M).mpr (sep_of_closure M h), fun h => closure_of_sep M
184     ((ra_comp_is_sep M).mp h)),
185    ra_comp_is_sep M)
186
187 /-! ## VII · The Williams route, wired to the machine -/
188
189 /- THE ROUTE ON THE MACHINE. 'Savings C' is a white-box satisfiability algorithm
190 for circuit class C beating 2^n by
191 a superpolynomial factor; 'williams' is Williams' theorem, savings for C give
192 NEXP ≠ C, carried; 'Ppoly' is the
193 class of polynomial-size circuits; 'np_not_ppoly_gives_sep' is the elementary
194 inclusion P ≤ P/poly, carried, by
195 which NP ≠ P/poly gives the machine's own separation. -/
196 structure Route (M : Machine) where
197   Class : Type
198   Savings : Class → Prop
199   NEXNotIn : Class → Prop
200   williams : ∀ C, Savings C → NEXNotIn C
201   Ppoly : Class
202   NNotPpoly : Prop
203   np_not_ppoly_gives_sep : NNotPpoly → Sep M
204
205 /- THE ROUTE REACHES: savings for a class give NEXP ≠ that class. -/
206 theorem route_reaches (M : Machine) (R : Route M) (C : R.Class) (s : R.Savings C) :
207   R.NEXNotIn C :=
208     R.williams C s
209
210 /- THE ROUTE CLOSES ON THE MACHINE: savings for general circuits and the scale-down
211 from NEXP to NP give the
212 machine's own separation. -/
213 theorem route_closes_on_the_machine (M : Machine) (R : Route M) (s : R.Savings
214   R.Ppoly)
215   (scaleDown : R.NEXNotIn R.Ppoly → R.NNotPpoly) : Sep M :=
216   (scaleDown (R.NEXNotIn R.Ppoly → R.NNotPpoly) (s : R.Savings R.Ppoly s))
217
218 /- THE ROUTE FILLS THE POSIT: supplied its two objects, the route fills least
219 escape itself. -/
220 theorem the_route_fills_least_escape (M : Machine) (R : Route M) (s : R.Savings
221   R.Ppoly)
222   (scaleDown : R.NEXNotIn R.Ppoly → R.NNotPpoly) : LeastEscape M :=
223   (closure_of_sep M (route_closes_on_the_machine M R s scaleDown))
224
225 /-! ## V · The master seal -/
226
227 /- THE MASTER SEAL: least escape closes the separation; the two posits are one, and
228 the third name, RA read on
229 computation, with them; the throne's holder decides the separation; on least
230 escape the holder is stopped at
231 every bound; every prime's fibre is a copy of complementation; and the Williams
232 route, supplied its two objects,
233 closes the machine's own separation. -/
234 theorem the_master_seal (M : Machine) (T : Throne M) :
235   (NothingEscapes M → Sep M) ∧ ((Asymmetry → Sep M) → NothingEscapes M) ∧
236   (Sep M → ∃ c k, Escapes M T.holder c k) ∧
237   (∀ _ : LeastEscape M, ∀ c k, ∃ x, ¬ Clean M T.holder c k x) ∧
238   (∀ p, IsPrime p → ∀ x : Fibre p, toBit (swapF x) = !(toBit x)) ∧
239   (NothingEscapes M → RAcamp M) ∧
240   (∀ R : Route M, R.Savings R.Ppoly → (R.NEXNotIn R.Ppoly → R.NNotPpoly) → Sep
241    M) :=

```

```

223 (sep_of_closure M, the_two_posits_are_one M, the_holder_decides_the_separation M
224   T,
225   fun L => the_act_stops_the_holder M T L, fun p hp =>
226     (prime_fibre_is_complementation p hp).1,
227     (the_three_names_are_one M).2.1, fun R s d => route_closes_on_the_machine M R s
228     d)
229 end PNP.FORTIFY
230
231 /-! ## Cones, pinned as printed -/
232 /-- info: 'PNP.FORTIFY.sep_of_closure' does not depend on any axioms -/
233 #guard_msgs in #print axioms PNP.FORTIFY.sep_of_closure
234 /-- info: 'PNP.FORTIFY.closure_of_sep' depends on axioms: [propext,
235   Classical.choice, Quot.sound] -/
236 #guard_msgs in #print axioms PNP.FORTIFY.closure_of_sep
237 /-- info: 'PNP.FORTIFY.the_holder_decides_the_separation' does not depend on any
238   axioms -/
239 #guard_msgs in #print axioms PNP.FORTIFY.the_holder_decides_the_separation
240 /-- info: 'PNP.FORTIFY.the_act_stops_the_holder' does not depend on any axioms -/
241 #guard_msgs in #print axioms PNP.FORTIFY.the_act_stops_the_holder
242 /-- info: 'PNP.FORTIFY.destroyed_is_unrecoverable' depends on axioms: [propext] -/
243 #guard_msgs in #print axioms PNP.FORTIFY.destroyed_is_unrecoverable
244 /-- info: 'PNP.FORTIFY.hidden_is_recoverable' depends on axioms: [propext,
245   Quot.sound] -/
246 #guard_msgs in #print axioms PNP.FORTIFY.hidden_is_recoverable
247 /-- info: 'PNP.FORTIFY.asymmetry_holds' depends on axioms: [propext, Quot.sound] -/
248 #guard_msgs in #print axioms PNP.FORTIFY.asymmetry_holds
249 /-- info: 'PNP.FORTIFY.the_two_posits_are_one' depends on axioms: [propext,
250   Classical.choice, Quot.sound] -/
251 #guard_msgs in #print axioms PNP.FORTIFY.every_closing_premise_fills_both
252 /-- info: 'PNP.FORTIFY.prime_fibre' depends on axioms: [propext, Quot.sound] -/
253 #guard_msgs in #print axioms PNP.FORTIFY.prime_fibre
254 /-- info: 'PNP.FORTIFY.prime_off_seat' depends on axioms: [propext, Quot.sound] -/
255 #guard_msgs in #print axioms PNP.FORTIFY.prime_off_seat
256 /-- info: 'PNP.FORTIFY.prime_fibre_is_complementation' depends on axioms: [propext,
257   Quot.sound] -/
258 #guard_msgs in #print axioms PNP.FORTIFY.prime_fibre_is_complementation
259 /-- info: 'PNP.FORTIFY.ra_comp_is_sep' does not depend on any axioms -/
260 #guard_msgs in #print axioms PNP.FORTIFY.ra_comp_is_sep
261 /-- info: 'PNP.FORTIFY.the_three_names_are_one' depends on axioms: [propext,
262   Classical.choice, Quot.sound] -/
263 #guard_msgs in #print axioms PNP.FORTIFY.the_three_names_are_one
264 /-- info: 'PNP.FORTIFY.route_reaches' does not depend on any axioms -/
265 #guard_msgs in #print axioms PNP.FORTIFY.route_reaches
266 /-- info: 'PNP.FORTIFY.route_closes_on_the_machine' does not depend on any axioms -/
267 #guard_msgs in #print axioms PNP.FORTIFY.route_closes_on_the_machine
268 /-- info: 'PNP.FORTIFY.the_route_fills_least_escape' depends on axioms: [propext,
269   Classical.choice, Quot.sound] -/
270 #guard_msgs in #print axioms PNP.FORTIFY.the_route_fills_least_escape
271 /-- info: 'PNP.FORTIFY.the_master_seal' depends on axioms: [propext,
272   Classical.choice, Quot.sound] -/
273 #guard_msgs in #print axioms PNP.FORTIFY.the_master_seal

```

Appendix B. The Closure Kernel, PNP_Least_Escape.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the closure and its double defense, twenty-four theorems. SHA-256: cfd2d9d6ea7d4f7b03ae07dd5976539d1168ff3da3c9d024f3b19518044f33be

```

1 /-!
2 # PNP_Least_Escape.lean · APEX-PSP-PNP-SEED-02 · Least Escape, the seal and its
3   double defense, hardened: Omega and AEGIS of the row
4
5 The P versus NP twin of PSP-RH-SEAL-01's 'double_defense_hardenend'.
6
7 OMEGA, the formal gate. Nothing passes for the separation but least escape. The
8   socket of the act is the closure; every premise that forces the separation fills
9   the field of the act; every premise either forces the closure or has a twin
10  machine
11  on which it holds and the closure fails; a keyless premise forces nothing.
12
13 AEGIS, the actuation gate. Nothing passes against least escape but a computed
14   escape: an algorithm, a polynomial bound, and every run clean. A finite stock of
15   examinations decides nothing. The root, which every computation re-enacts, crosses
16   neither way. The refusal is the same under every logic.
17
18 THE COSMIC READING, at its exact scope. Every pair a substrate can examine lands
19   on
20   exactly one gate: a stopping instance, which confirms least escape at that pair,
21   or
22   a clean escape, which refutes it. There is no third landing. Every confirming
23   computation, anywhere, is an atomic witness of least escape at its pair and pays
24   the
25   floor; under the act, every pair confirms and no computation escapes. What no
26   computation does is enact the universal: the universal is the act.
27
28 ARMOR SCOPE. The defense forces the form of every objection and never the value:
29   the closure holds on one coherent machine and fails on another. Without the act
30   the
31   value is marked [.].

```

```

26 Core Lean 4, no import. Cones pinned at the foot, each as printed by the kernel.
27 -/
28
29 namespace PNP.HARDENED
30
31 /-! ## 0 · the machine, the closure, the act -/
32 structure Machine where
33   Alg : Type
34   Inst : Type
35   size : Inst → Nat
36   out : Alg → Inst → Bool
37   time : Alg → Inst → Nat
38   sat : Inst → Bool
39
40 def bound (c k n : Nat) : Nat := c * n ^ k + c
41 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
42   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
43 def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M A c k x
44 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
45 def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
46
47 structure LeastEscape (M : Machine) where
48   closure : NothingEscapes M
49
50 structure Examination (M : Machine) where
51   A : M.Alg
52   c : Nat
53   k : Nat
54   x : M.Inst
55   stops : ¬ Clean M A c k x
56
57 theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
58   fun (A, c, k, he) => match h A c k with
59   | (x, hx) => hx (he x)
60
61 theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
62   fun A c k => Classical.byContradiction fun hn =>
63     h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))
64
65 theorem sep_from_the_act (M : Machine) (L : LeastEscape M) : Sep M := sep_of_closure
66   M L.closure
67
68 /-! ## the two coherent machines -/
69 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
70 | 0 => rfl
71 | n + 1 => beq_self n
72
73 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
74   cases hb : Nat.beq x K with
75   | false => rfl
76   | true =>
77     have e : x = K := Nat.eq_of_beq_eq_true hb
78     rw [e] at h; exact absurd h (Nat.lt_irrefl K)
79
80 def stageWorld (n : Nat) : Machine :=
81   (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A n, fun _ _ => 0, fun _ _ => true)
82 def closedWorld : Machine :=
83   (Nat, Unit, fun _ => 0, fun _ _ => false, fun _ _ => 0, fun _ _ => true)
84
85 theorem stage_escapes (n : Nat) : Escapes (stageWorld n) n 0 0 :=
86   fun _ => (beq_self n, Nat.zero_le _)
87 theorem closed_world_closes : NothingEscapes closedWorld :=
88   fun _ _ => ((), fun (h, _) => Bool.noConfusion h)
89 theorem stage_fails_closure (n : Nat) : ¬ NothingEscapes (stageWorld n) :=
90   fun h => sep_of_closure _ h (n, 0, 0, stage_escapes n)
91
92 /-! ## I · OMEGA, the formal gate -/
93
94 /-- The socket of the act is the closure. -/
95 theorem socket_is_the_value (M : Machine) : Nonempty (LeastEscape M) ↔
96   NothingEscapes M :=
97   (fun (L) => L.closure, fun h => {(h)})
98
99 /-- Every premise that forces the separation fills the field of the act. -/
100 theorem forcing_premise_fills_the_act (M : Machine) (Q : Prop) (h : Q → Sep M) :
101   Q → Nonempty (LeastEscape M) :=
102   fun q => {(closure_of_sep M (h q))}
103
104 def Forces (P : Machine → Prop) : Prop := ∀ M, P M → NothingEscapes M
105
106 /-- Every premise forces the closure or has a twin machine where it holds and the
107   closure fails. -/
108 theorem forces_or_has_a_twin (P : Machine → Prop) :
109   Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M :=
110   Classical.byCases (fun h : Forces P => 0r.inl h) (fun h => 0r.inr h)
111   (Classical.byContradiction fun hn => h fun M hp =>
112     Classical.byContradiction fun hc => hn (M, hp, hc)))
113
114 def Keyless (P : Machine → Prop) : Prop := ∀ M, P M
115
116 /-- A keyless premise forces nothing. -/
117 theorem keyless_forces_nothing (P : Machine → Prop) (hp : Keyless P) : ¬ Forces P :=
118   fun h => stage_fails_closure 0 (h (stageWorld 0) (hp _))
119
120 /-! ## II · AEGIS, the actuation gate -/
121
122 /-- A computed escape refutes the closure. -/
123 theorem escape_refutes (M : Machine) (A : M.Alg) (c k : Nat) (h : Escapes M A c k) :
124   ¬ NothingEscapes M :=
125   fun hc => sep_of_closure M hc (A, c, k, h)
126
127 /-- Nothing passes against the closure but a computed escape. -/
128 theorem rejection_is_a_witness (M : Machine) :
129   ¬ NothingEscapes M → ∃ A c k, Escapes M A c k :=
130   (fun h => Classical.byContradiction fun hn => h (closure_of_sep M hn),
131     fun (A, c, k, he) => escape_refutes M A c k he)

```

```

130 /-- A finite stock of examinations decides nothing. -/
131 theorem record_decides_nothing (n : Nat) :
132   (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k ()) A ¬ NothingEscapes
133   (stageWorld n) :=
134   (fun A _ _ ha (h, _) => by
135     have h' : Nat.beq A n = true := h
136     rw [beq_false_of_lt A n ha] at h'
137     exact Bool.noConfusion h',
138     stage_fails_closure n)
139
140 /-- The root, which every computation re-enacts, crosses neither way. -/
141 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
142
143 theorem root_crosses_neither_way :
144   (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → NothingEscapes
145     M) A
146   (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → ¬
147     NothingEscapes M) :=
148   (fun h => stage_fails_closure 0 (h (stageWorld 0) (fun _ _ => trivial))),
149   fun h => h closedWorld (fun _ _ => trivial) closed_world_closes)
150
151 /-- The refusal is constant under every logic. -/
152 def refusal (_logic : String) : String :=
153   "nothing passes against least escape but a computed escape"
154
155 theorem aegis_constant (l₁ l₂ : String) : refusal l₁ = refusal l₂ := rfl
156
157 /-- ## III · the cosmic reading, at its exact scope -/
158
159 /-- Every examinable pair lands on exactly one gate: a stopping instance or a clean
160 escape. -/
161 theorem every_pair_lands (M : Machine) (A : M.Alg) (c k : Nat) :
162   ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) A
163   ¬ ((∃ x, ¬ Clean M A c k x) ∧ Escapes M A c k) :=
164   (Classical.byCases (fun h : Escapes M A c k => Or.inr h)
165     (fun h => Or.inl (Classical.byContradiction fun hn =>
166       h fun x => Classical.byContradiction fun hx => hn (x, hx))),
167     fun {(x, hx), he} => hx (he x))
168
169 /-- A confirming computation is an atomic witness of least escape at its pair. -/
170 theorem confirmation_is_atomic (M : Machine) (e : Examination M) :
171   (∃ x, ¬ Clean M e.A e.c e.k x) A ¬ Escapes M e.A e.c e.k :=
172   ((e.x, e.stops), fun he => e.stops (he e.x))
173
174 /-- Every confirming computation pays the floor. -/
175 def examCost (floor : Nat) (_e : Examination M) : Nat := floor * 1
176
177 theorem confirmation_pays (M : Machine) (floor : Nat) (h : 0 < floor) (e :
178   Examination M) :
179   0 < examCost floor e :=
180   Nat.mul_pos h (Nat.zero_lt_one)
181
182 /-- Under the act, every pair confirms and no computation escapes. -/
183 theorem act_leaves_no_escape (M : Machine) (L : LeastEscape M) :
184   (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) A
185   (∀ A c k, ¬ Escapes M A c k) :=
186   (fun A c k => match L.closure A c k with
187     | (x, hx) => ((A, c, k, x, hx), rfl, rfl),
188     fun A c k he => sep_from_the_act M L (A, c, k, he))
189
190 /-- ## IV · armor scope and silence -/
191 theorem armor_scope : NothingEscapes closedWorld A ¬ NothingEscapes (stageWorld 0)
192   :=
193   (closed_world_closes, stage_fails_closure 0)
194
195 inductive Token : Type
196 | sealSep | dot
197 deriving DecidableEq, Repr
198
199 /-- The emission: with the act supplied, the seal; without it, the dot. -/
200 def verdict (actSupplied : Bool) : Token := if actSupplied then .sealSep else .dot
201
202 theorem value_marked_dot : verdict false = .dot ∧ verdict true = .sealSep := (rfl,
203   rfl)
204
205 /-- ## V · THE DOUBLE DEFENSE, HARDENED -/
206 theorem double_defense_harden :
207   -- the act carries the separation
208   (∀ (M : Machine), LeastEscape M → Sep M) A
209   -- OMEGA
210   (∀ (M : Machine, Nonempty (LeastEscape M) → NothingEscapes M) A
211     (∀ (M : Machine) (Q : Prop), (Q → Sep M) → Q → Nonempty (LeastEscape M)) A
212     (∀ P : Machine → Prop, Forces P ∨ ∃ M, P M A → NothingEscapes M) A
213     (∀ P : Machine → Prop, Keyless P → ¬ Forces P) A
214     -- AEGIS
215     (∀ M : Machine, ¬ NothingEscapes M → ∃ A c k, Escapes M A c k) A
216     (∀ n : Nat, (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k ())) A ¬
217     NothingEscapes (stageWorld n)) A
218     ((¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) →
219       NothingEscapes M) A
220       (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → ¬
221         NothingEscapes M)) A
222     (∀ l₁ l₂ : String, refusal l₁ = refusal l₂) A
223     -- the cosmic reading
224     (∀ (M : Machine) (A : M.Alg) (c k : Nat),
225       ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) A ¬ ((∃ x, ¬ Clean M A c k x) A
226         Escapes M A c k)) A
227     (∀ (M : Machine) (e : Examination M), (∃ x, ¬ Clean M e.A e.c e.k x) A ¬ Escapes
228       M e.A e.c e.k) A
229     (∀ (M : Machine), LeastEscape M →
230       (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) A (∀ A c k, ¬
231         Escapes M A c k)) A
232     -- armor scope and silence
233     (NothingEscapes closedWorld A → NothingEscapes (stageWorld 0)) A
234     (verdict false = .dot ∧ verdict true = .sealSep) :=
235     (sep_from_the_act, socket_is_the_value, forcing_premise_fills_the_act,
236       forces_or_has_a_twin,
```

```

224   keyless_forces_nothing, rejection_is_a_witness, record_decides_nothing,
225   root_crosses_neither_way,
226   aegis_constant, every_pair_lands, confirmation_is_atomic, act_leaves_no_escape,
227   armor_scope,
228   value_marked_dot)
229
230 end PNP.HARDENED
231
232 /--! ## Cones, pinned as printed -/
233 /-- info: 'PNP.HARDENED.sep_of_closure' does not depend on any axioms -/
234 #guard_msgs in #print axioms PNP.HARDENED.sep_of_closure
235 /-- info: 'PNP.HARDENED.closure_of_sep' depends on axioms: [propext,
236   Classical.choice, Quot.sound] -/
237 #guard_msgs in #print axioms PNP.HARDENED.closure_of_sep
238 /-- info: 'PNP.HARDENED.sep_from_the_act' does not depend on any axioms -/
239 #guard_msgs in #print axioms PNP.HARDENED.sep_from_the_act
240 /-- info: 'PNP.HARDENED.beq_self' does not depend on any axioms -/
241 #guard_msgs in #print axioms PNP.HARDENED.beq_self
242 /-- info: 'PNP.HARDENED.beq_false_of_lt' does not depend on any axioms -/
243 #guard_msgs in #print axioms PNP.HARDENED.beq_false_of_lt
244 /-- info: 'PNP.HARDENED.stage_escapes' does not depend on any axioms -/
245 #guard_msgs in #print axioms PNP.HARDENED.stage_escapes
246 /-- info: 'PNP.HARDENED.closed_world_closes' does not depend on any axioms -/
247 #guard_msgs in #print axioms PNP.HARDENED.closed_world_closes
248 /-- info: 'PNP.HARDENED.stage_fails_closure' does not depend on any axioms -/
249 #guard_msgs in #print axioms PNP.HARDENED.stage_fails_closure
250 /-- info: 'PNP.HARDENED.socket_is_the_value' does not depend on any axioms -/
251 #guard_msgs in #print axioms PNP.HARDENED.socket_is_the_value
252 /-- info: 'PNP.HARDENED.forcing_premise_fills_the_act' depends on axioms: [propext,
253   Classical.choice, Quot.sound] -/
254 #guard_msgs in #print axioms PNP.HARDENED.forcing_premise_fills_the_act
255 /-- info: 'PNP.HARDENED.forces_or_has_a_twin' depends on axioms: [propext,
256   Classical.choice, Quot.sound] -/
257 #guard_msgs in #print axioms PNP.HARDENED.forces_or_has_a_twin
258 /-- info: 'PNP.HARDENED.keyless_forces_nothing' does not depend on any axioms -/
259 #guard_msgs in #print axioms PNP.HARDENED.keyless_forces_nothing
260 /-- info: 'PNP.HARDENED.escape_refutes' does not depend on any axioms -/
261 #guard_msgs in #print axioms PNP.HARDENED.escape_refutes
262 /-- info: 'PNP.HARDENED.rejection_is_a_witness' depends on axioms: [propext,
263   Classical.choice, Quot.sound] -/
264 #guard_msgs in #print axioms PNP.HARDENED.rejection_is_a_witness
265 /-- info: 'PNP.HARDENED.record_decides_nothing' does not depend on any axioms -/
266 #guard_msgs in #print axioms PNP.HARDENED.record_decides_nothing
267 /-- info: 'PNP.HARDENED.root_crosses_neither_way' does not depend on any axioms -/
268 #guard_msgs in #print axioms PNP.HARDENED.root_crosses_neither_way
269 /-- info: 'PNP.HARDENED.aegis_constant' does not depend on any axioms -/
270 #guard_msgs in #print axioms PNP.HARDENED.aegis_constant
271 /-- info: 'PNP.HARDENED.every_pair_lands' depends on axioms: [propext,
272   Classical.choice, Quot.sound] -/
273 #guard_msgs in #print axioms PNP.HARDENED.every_pair_lands
274 /-- info: 'PNP.HARDENED.confirmation_is_atomic' does not depend on any axioms -/
275 #guard_msgs in #print axioms PNP.HARDENED.confirmation_is_atomic
276 /-- info: 'PNP.HARDENED.confirmation_pays' does not depend on any axioms -/
277 #guard_msgs in #print axioms PNP.HARDENED.confirmation_pays
278 /-- info: 'PNP.HARDENED.act_leaves_no_escape' does not depend on any axioms -/
279 #guard_msgs in #print axioms PNP.HARDENED.act_leaves_no_escape
280 /-- info: 'PNP.HARDENED.armor_scope' does not depend on any axioms -/
281 #guard_msgs in #print axioms PNP.HARDENED.armor_scope
282 /-- info: 'PNP.HARDENED.value_marked_dot' does not depend on any axioms -/
283 #guard_msgs in #print axioms PNP.HARDENED.value_marked_dot
284 /-- info: 'PNP.HARDENED.double_defense_harden' depends on axioms: [propext,
285   Classical.choice, Quot.sound] -/
286 #guard_msgs in #print axioms PNP.HARDENED.double_defense_harden
```

Appendix C. The Frontier Kernel, PNP_Frontier_II.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the ground, the reading, the vanishing seat, the frontier and the route, one hundred thirty-six theorems. SHA-256: 960b09ba71219e4c95064a5a6a4bb002a622895b46bbaedacc3b509c11a9cd51

```

1 /-
2   PNP_Frontier_II.lean · APEX-PSP-PNP-SEED-02 · Nothing Escapes · the frontier of
3   the act
4   Eleven sections, each a kernel of the 4 October 2026 forge, joined in one file:
5   the kinetic face,
6   the Arrhenius scope, the block chain on the Empty Throne, the vanishing G0L, the
7   missing object,
8   the prime bridge, the blocker and round 2, the route of RA and Freedom, the
9   three-seal lock, the
10  deed as supply, and the identification check. Core Lean 4, no import, no axiom
11  declared, no sorry.
12  Every cone is pinned beside its section.
13 -/
14
15 namespace PNP.KINETIC
16
17 /-- The kinetic frame. Physics enters as fields, never as 'axiom':
18 'floor' is the Landauer floor per irreversible op, in quanta;
19 'budget' is the Bekenstein-capped resource of the region, in quanta. -/
20 structure Frame where
21   floor      : Nat
22   floor_pos  : 0 < floor
23   budget     : Nat
24
25 /-- Energy of a process of `ops` irreversible registrations. -/
```

```

21 def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops
22
23 /-- L2 face. No completed irreversible registration is free. -/
24 theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) :
25   0 < cost F k :=
26   Nat.mul_pos F.floor_pos hk
27
28 /-- Transport. The energy order is exactly the op order. -/
29 theorem cost_lt_iff (F : Frame) (a b : Nat) :
30   cost F a < cost F b ↔ a < b :=
31   (fun h => match Nat.lt_or_ge a b with
32     | Or.inl hl => hl
33     | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left F.floor hge)),
34   fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos)
35
36 /-- L_bf. Enumerating 2^N configurations at N = budget exceeds the budget. -/
37 theorem brute_force_exhausts (F : Frame) :
38   F.budget < cost F (2 ^ F.budget) :=
39   Nat.lt_of_lt_of_le Nat.lt_two_pow_self
40   (Nat.le_mul_of_pos_left (2 ^ F.budget) F.floor_pos)
41
42 /-- Monotone form: every N past the budget also exhausts it. -/
43 theorem brute_force_exhausts_beyond (F : Frame) (N : Nat) (h : F.budget ≤ N) :
44   F.budget < cost F (2 ^ N) :=
45   Nat.lt_of_lt_of_le (brute_force_exhausts F)
46   (Nat.mul_le_mul_left (Nat.pow_le_pow_right (by decide) h))
47
48 /-- L1, scoped to the enumerative generator. Given the op gap v < 2^N,
49 the kinetic gap follows. The op gap is a hypothesis, not a product. -/
50 theorem enumerative_asymmetry (F : Frame) (N v : Nat) (h : v < 2 ^ N) :
51   cost F v < cost F (2 ^ N) :=
52   (cost_lt_iff F v (2 ^ N)).mpr h
53
54 /-- The ceiling. For every generator op-count g and verifier op-count v,
55 the kinetic separation holds in a frame iff the op separation holds.
56 The frame adds nothing to the formal question, so L3 enters only as a
57 hypothesis on g and is never decided here. -/
58 theorem kinetic_faithful (F : Frame) (g v : Nat → Nat) :
59   (∀ N, cost F (v N) < cost F (g N)) ↔ (∀ N, v N < g N) :=
60   (fun h N => (cost_lt_iff F _ _).mpr (h N),
61   fun h N => (cost_lt_iff F _ _).mpr (h N))
62
63 /-- Frame-independence. The op separation reads the same in any two frames. -/
64 theorem frame_independent (F F2 : Frame) (g v : Nat → Nat) :
65   (∀ N, cost F1 (v N) < cost F1 (g N)) ↔ (∀ N, cost F2 (v N) < cost F2 (g N)) :=
66   (kinetic_faithful F1 g v).trans (kinetic_faithful F2 g v).symm
67
68 /-- The canonical frame, one quantum per op, one quantum of budget: the
69 battery instance, by evaluation. -/
70 def unitFrame : Frame := (1, by decide, 1)
71
72 theorem unit_battery :
73   cost unitFrame (2 ^ 1) = 2 ∧ unitFrame.budget < cost unitFrame (2 ^
74   unitFrame.budget) := by decide
75
76 end PNP.KINETIC
77
78 /-- info: 'PNP.KINETIC.no_free_registration' does not depend on any axioms -/
79 #guard_msgs in #print axioms PNP.KINETIC.no_free_registration
80 /-- info: 'PNP.KINETIC.cost_lt_iff' does not depend on any axioms -/
81 #guard_msgs in #print axioms PNP.KINETIC.cost_lt_iff
82 /-- info: 'PNP.KINETIC.brute_force_exhausts' does not depend on any axioms -/
83 #guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts
84 /-- info: 'PNP.KINETIC.brute_force_exhausts_beyond' does not depend on any axioms -/
85 #guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts_beyond
86 /-- info: 'PNP.KINETIC.enumerative_asymmetry' does not depend on any axioms -/
87 #guard_msgs in #print axioms PNP.KINETIC.enumerative_asymmetry
88 /-- info: 'PNP.KINETIC.kinetic_faithful' does not depend on any axioms -/
89 #guard_msgs in #print axioms PNP.KINETIC.kinetic_faithful
90 /-- info: 'PNP.KINETIC.frame_independent' does not depend on any axioms -/
91 #guard_msgs in #print axioms PNP.KINETIC.frame_independent
92 /-- info: 'PNP.KINETIC.unit_battery' does not depend on any axioms -/
93 #guard_msgs in #print axioms PNP.KINETIC.unit_battery
94
95 /-!
96 # PNP.ARRHENIUS · the kinetic proof of the April 2026 P vs NP paper, formalized
97
98 Source: Islam, "Geometric Determination of P vs NP with Omega Seal", v10.0,
99 10.5281/zenodo.19588804, Part VII, "The Spin-Glass Isomorphism".
100 The paper's kinetic step: ground states of a frustrated glass sit behind
101 barriers ΔE growing with N; Arrhenius gives escape time exp(ΔE/KT);
102 so generation is exponential while verification is linear.
103
104 Part I seats that step at its true scope, with physics as fields, never `axiom`.
105 Part II executes the scope fence: on one fixed problem, the barrier exists
106 under one move set and vanishes under another. A barrier is a property of a
107 dynamics, never of a problem. No custom axiom; no core axiom either.
108 -/
109
110 namespace PNP.ARRHENIUS
111
112 /-! ## I · The kinetic step, at its scope: thermally activated relaxation -/
113
114 /-- A relaxation frame. `barrier N` is ΔE/KT in bits at size N; `grows` is the
115 paper's "barriers scale with N"; `arrhenius` is t ≥ exp(ΔE/KT), base two. -/
116 structure Frame where
117   barrier : Nat → Nat
118   grows : ∀ N, N ≤ barrier N
119   escape : Nat → Nat
120   arrhenius : ∀ N, 2 ^ barrier N ≤ escape N
121
122 /-- Relaxation over a growing barrier takes exponential time. -/
123 theorem relaxation_exponential (F : Frame) (N : Nat) (2 ^ N ≤ F.escape N :=
124   Nat.le_trans (Nat.pow_le_pow_right (by decide) (F.grows N)) (F.arrhenius N))
125
126 /-- At size N = B the relaxation time exceeds any budget B. -/
127 theorem relaxation_exhausts (F : Frame) (B : Nat) : B < F.escape B :=

```

```

127   Nat.lt_of_lt_of_le Nat.lt_two_pow_self (relaxation_exponential F B)
128
129 /-! ## II · The scope fence, executed on one XORSAT instance
130 Three variables x y z, three parity constraints, all with target 0:
131 z = 0, y ⊕ z = 0, x ⊕ z = 0. Energy counts violated constraints. -/
132
133 abbrev S := Bool × Bool × Bool
134
135 def states : List S :=
136   [false, true].flatMap fun a => [false, true].flatMap fun b =>
137   [false, true].map fun c => (a, b, c)
138
139 def viol (x : S) : List Bool := [x.2.2, xor x.2.1 x.2.2, xor x.1 x.2.2]
140
141 def energy (x : S) : Nat := ((viol x).filter id).length
142
143 def move (x m : S) : S := (xor x.1 m.1, xor x.2.1 m.2.1, xor x.2.2 m.2.2)
144
145 /-- The glass dynamics: single-spin flips. -/
146 def flips : List S := [(true, false, false), (false, true, false), (false, false,
147   true)]
148
149 /-- The elimination dynamics: columns of A-1 over GF(2). Each move toggles
150 exactly one constraint. Computing them is Gaussian elimination, polynomial time. -/
151 def elimMoves : List S := [(true, true, true), (false, true, false), (true, false,
152   false)]
153
154 def ground : S := (false, false, false)
155 def trap : S := (true, true, true)
156
157 theorem eight_states : states.length = 8 := by decide
158
159 theorem ground_has_zero_energy : energy ground = 0 := by decide
160
161 /-- Under single flips the trap is a strict local minimum above the ground:
162 a barrier, exactly the glass picture of the paper. -/
163 theorem trap_is_strict_local_min :
164   energy trap = 1 ∧ flips.all (fun m => decide (energy trap < energy (move trap
165   m))) = true := by
166   decide
167
168 /-- Each elimination move toggles exactly one constraint. -/
169 theorem elim_moves_toggle_one :
170   states.all (fun x => elimMoves.all (fun m =>
171     decide ((List.zipWith (fun a b => a != b) (viol x) (viol (move x m))).count
172     true = 1))) = true := by
173   decide
174
175 /-- Under elimination moves no state above the ground is a local minimum:
176 the landscape has no barrier at all. -/
177 theorem elim_landscape_barrier_free :
178   states.all (fun x => decide (energy x = 0) ||
179     elimMoves.any (fun m => decide (energy (move x m) < energy x))) = true := by
180   decide
181
182 /-- The fence. One problem, one energy function, two dynamics: a barrier under
183 the first and none under the second. The barrier belongs to the dynamics. -/
184 theorem barrier_belongs_to_the_dynamics :
185   (energy trap > energy ground ∧
186     flips.all (fun m => decide (energy trap < energy (move trap m))) = true) ∧
187   states.all (fun x => decide (energy x = 0) ||
188     elimMoves.any (fun m => decide (energy (move x m) < energy x))) = true :=
189   (by decide, trap_is_strict_local_min.2), elim_landscape_barrier_free)
190
191 end PNP.ARRHENIUS
192
193 /-- info: 'PNP.ARRHENIUS.relaxation_exponential' does not depend on any axioms -/
194 #guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exponential
195 /-- info: 'PNP.ARRHENIUS.relaxation_exhausts' does not depend on any axioms -/
196 #guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exhausts
197 /-- info: 'PNP.ARRHENIUS.eight_states' does not depend on any axioms -/
198 #guard_msgs in #print axioms PNP.ARRHENIUS.eight_states
199 /-- info: 'PNP.ARRHENIUS.ground_has_zero_energy' does not depend on any axioms -/
200 #guard_msgs in #print axioms PNP.ARRHENIUS.ground_has_zero_energy
201 /-- info: 'PNP.ARRHENIUS.trap_is_strict_local_min' does not depend on any axioms -/
202 #guard_msgs in #print axioms PNP.ARRHENIUS.trap_is_strict_local_min
203 /-- info: 'PNP.ARRHENIUS.elim_moves_toggle_one' does not depend on any axioms -/
204 #guard_msgs in #print axioms PNP.ARRHENIUS.elim_moves_toggle_one
205 /-- info: 'PNP.ARRHENIUS.elim_landscape_barrier_free' does not depend on any axioms -/
206 #guard_msgs in #print axioms PNP.ARRHENIUS.elim_landscape_barrier_free
207 /-- info: 'PNP.ARRHENIUS.barrier_belongs_to_the_dynamics' does not depend on any
208 axioms -/
209 #guard_msgs in #print axioms PNP.ARRHENIUS.barrier_belongs_to_the_dynamics
210
211 /-!
212 # PNP.CHAIN · the block chain of the Mother Codex on P versus NP, closed on the
213 Empty Throne as [], then joined to the kinetic face as the asymmetry chain
214
215 Source of the blocks: Geometric Mother Codex v3.46.0 (10.5281/zenodo.23117445),
216 P-ABSOLUTE-PNP-1, the eight absolute barriers A1-A8; P-PNP-COMP-2, the composite
217 verdict [κ] · [κ T] · [·]; FOUNDATION-01, the Empty Throne.
218 Source of the kinetic face: PNP_Kinetic.lean and PNP_Arrhenius.lean (this session).
219
220 Every theorem below is pinned at the foot: no custom axiom, and no core axiom
221 either, not propext, not Quot.sound, not Classical.choice. Physics enters only
222 as structure fields, so the premise is visible in each statement.
223 Lean 4 core, no import.
224
225 namespace PNP.CHAIN
226
227 /-! ## 0 · The verdict economy and the dot -/
228
229 inductive Token : Type
230 | seal | broken | opn | dot
231 deriving DecidableEq, Repr

```

```

228
229 /-- Three states are the economy. The dot stands outside it. -/
230 def Token.inEconomy : Token → Bool
231 | .dot => false
232 | _ => true
233
234 /-- ## I · The eight blocks, one leg each -/
235 namespace BLOCKS
236
237 /-- A1, the α-class closure. Complementation has no fixed point. -/
238 theorem a1_no_seat : ∀ b : Bool, (!b) ≠ b
239 | true => Bool.noConfusion
240 | false => Bool.noConfusion
241
242 /-- A1, carrier form. A symmetry that fixes a state carries no anti-invariant
243 reading, so no fixed-locus instrument transfers onto complementation. -/
244 theorem a1_no_carrier {S : Type} (τ : S → S) (s₀ : S) (hfix : τ s₀ = s₀) :
245   ¬ ∃ φ : S → Bool, ∀ x, φ (τ x) = !φ x := by
246   intro (h, φ)
247   have h := hφ s₀
248   rw [hfix] at h
249   cases e : φ s₀ with
250   | true => rw [e] at h; exact Bool.noConfusion h
251   | false => rw [e] at h; exact Bool.noConfusion h
252
253 /-- A2, orientation-blindness. A lock invariant under the flip of the sign
254 cannot be decoded into the sign. -/
255 theorem a2_orientation_blind {Q X : Type} (L : Bool × Q → X)
256   (hL : ∀ b q, L (b, q) = L (!b, q)) (q : Q) :
257   ¬ ∃ dec : X → Bool, ∀ w, s, dec (L s) = s.1 := by
258   intro (dec, h)
259   have h1 := h (true, q)
260   have h2 := h (false, q)
261   rw [hL true q] at h1
262   exact Bool.noConfusion (h1.symm.trans h2)
263
264 /-- A3, the physical block. A record shared by two worlds that differ on the
265 separation decides nothing; no reading of the record returns the separation. -/
266 theorem a3_record_decides_nothing {W R : Type} (rec : W → R) (sep : W → Bool)
267   (w₁ w₂ : W) (hr : rec w₁ = rec w₂) (hs : sep w₁ ≠ sep w₂) :
268   ¬ ∃ g : R → Bool, ∀ w, g (rec w) = sep w := by
269   intro (g, hg)
270   apply hs
271   rw [~ hg w₁, ~ hg w₂, hr]
272
273 /-- A4, the misattribution bar. The root's presence is constant across both
274 worlds, and a constant reading carries no bit about the separation. -/
275 theorem a4_constant_reads_nothing (c : Bool) (s₁ s₂ : Bool) (hs : s₁ ≠ s₂) :
276   ¬ (c = s₁ ∧ c = s₂) :=
277   fun (h1, h2) => hs (h1.symm.trans h2)
278
279 /-- A5, the rung law, structural face. Each rung's consistency is owed one rung
280 up; no rung discharges its own, and the regress has no terminal rung.
281 The Gödel content is cited (Turing 1939, Feferman 1962), not proved here. -/
282 def owed (n : Nat) : Nat := n + 1
283
284 theorem a5_rung_law : (∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m) := by
285   refine (fun n h => ?_, fun n => (n + 1, Nat.lt_succ_self n))
286   have : n < owed n := Nat.lt_succ_self n
287   rw [h] at this
288   exact Nat.lt_irrefl n this
289
290 /-- A6, the aperture machine. Eight verdict-side actions and one supply. -/
291 inductive Act : Type
292 | exertion | extrapolation | wanting | method | metatheory
293 | evidence | architecture | recoverer | supply
294 deriving DecidableEq, Repr
295
296 def Act.isSupply : Act → Bool
297 | .supply => true
298 | _ => false
299
300 def step (a : Act) (s : Bool) : Bool := s || a.isSupply
301
302 def verdictSide : List Act :=
303   [.exertion, .extrapolation, .wanting, .method, .metatheory,
304   .evidence, .architecture, .recoverer]
305
306 theorem a6_aperture :
307   verdictSide.length = 8 ∧
308   verdictSide.all (fun a => step a false == false) = true ∧
309   step .supply false = true := by
310   decide
311
312 /-- A7, reflexive closure. The certify-completely map strictly increases,
313 ten to twenty in five steps, and has no fixed point. -/
314 def certify (n : Nat) : Nat := n + 2
315
316 def iter : Nat → Nat → Nat
317 | 0, n => n
318 | k + 1, n => iter k (certify n)
319
320 theorem a7_reflexive_closure : iter 5 10 = 20 ∧ ∀ n, certify n ≠ n := by
321   refine (rfl, fun n h => ?_)
322   have : n < certify n := Nat.lt_succ_of_lt (Nat.lt_succ_self n)
323   rw [h] at this
324   exact Nat.lt_irrefl n this
325
326 /-- A8, the soundness cell. A sound prover never holds both the route and
327 the wall against it, so sighted-and-walled is empty. -/
328 theorem a8_soundness_cell (Prov : Prop → Prop) (sound : ∀ p, Prov p → p) (P : Prop) :
329   ¬ (Prov P ∧ Prov (¬ P)) :=
330   fun (h1, h2) => sound _ h2 (sound _ h1)
331
332 /-- The canonical two worlds of the record: one physics ledger, two values. -/
333 def equalWorld : Nat × Bool := (1, false)

```

```

334 def separatedWorld : Nat × Bool := (1, true)
335
336 /-- The block chain, every link in one statement. -/
337 theorem block_chain :
338   (∀ b : Bool, (!b) ≠ b) ∧
339   (¬ ∃ φ : Bool → Bool, ∀ x, φ (id x) = !φ x) ∧
340   (¬ ∃ dec : Bool → Bool, ∀ s : Bool × Unit, dec ((fun _ => true) s) = s.1) ∧
341   (¬ ∃ g : Nat → Bool, ∀ w : Nat × Bool,
342     (w = equalWorld v w = separatedWorld) → g w.1 = w.2) ∧
343   (¬ (true = equalWorld.2 ∧ true = separatedWorld.2)) ∧
344   ((∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m)) ∧
345   (verdictSide.length = 8 ∧
346     verdictSide.all (fun a => step a false == false) = true ∧
347     step .supply false = true) ∧
348   (iter 5 10 = 20 ∧ ∀ n, certify n ≠ n) ∧
349   (∀ (Prov : Prop → Prop), (∀ p, Prov p → p) → ∀ P : Prop, ¬ (Prov P ∧ Prov (¬
350     P))) := by
351   refine (a1_no_seat, a1_no_carrier id true rfl,
352     a2_orientation_blind (fun _ => true) (fun _ => rfl) (),
353     ?_, a4_constant_reads_nothing true false true Bool.noConfusion,
354     a5_rung_law, a6_aperture, a7_reflexive_closure, a8_soundness_cell)
355   intro (g, hg)
356   have h1 := hg equalWorld (0r.inl rfl)
357   have h2 := hg separatedWorld (0r.inr rfl)
358   exact Bool.noConfusion (h1.symm.trans h2)
359
360 end BLOCKS
361
362 /-- ## II · The Empty Throne, and the dot -/
363 namespace THRONE
364
365 def allFixed : List Bool → Bool
366 | [] => true
367 | b :: bs => (!b) == b) && allFixed bs
368
369 /-- The seat is vacant. Least erasure on complementation holds of the empty
370 configuration only: a fixed-point-free fold selects nothing. -/
371 theorem seat_vacant : ∀ l : List Bool, allFixed l = true → l = []
372 | [], _ => rfl
373 | true :: _, h => Bool.noConfusion h
374 | false :: _, h => Bool.noConfusion h
375
376 /-- The throne is empty. Nothing below proves a posited root in general:
377 the root is premise-grade by theorem. -/
378 theorem throne_empty : ¬ ∀ RA : Prop, RA :=
379   fun h => h False
380
381 /-- The root forces no value. No premise of existence, however strong, forces
382 an arbitrary conclusion: the separation is not climbed to from the root. -/
383 theorem root_forces_no_value : ¬ ∀ RA Sep : Prop, RA → Sep :=
384   fun h => h True False trivial
385
386 /-- The chain's terminal emission. -/
387 def chainVerdict : Token := .dot
388
389 theorem dot_is_final : chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
390   (rfl, rfl)
391
392 /-- The block chain terminated on the Empty Throne as [:]: every block holds,
393 the seat is vacant, the throne is empty, the root forces no value, and the
394 emission is the dot, outside the economy, owing nothing. -/
395 theorem block_chain_terminal :
396   (∀ b : Bool, (!b) ≠ b) ∧
397   (∀ l : List Bool, allFixed l = true → l = []) ∧
398   (¬ ∀ RA : Prop, RA) ∧
399   (¬ ∀ RA Sep : Prop, RA → Sep) ∧
400   chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
401   (BLOCKS.block_chain.1, seat_vacant, throne_empty, root_forces_no_value,
402     dot_is_final.1, dot_is_final.2)
403
404 end THRONE
405
406 /-- ## III · The kinetic face -/
407 namespace KINETIC
408
409 structure Frame where
410   floor : Nat
411   floor_pos : 0 < floor
412   budget : Nat
413
414 def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops
415
416 theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) : 0 < cost F k :=
417   Nat.mul_pos F.floor_pos hk
418
419 theorem cost_lt_iff (F : Frame) (a b : Nat) : cost F a < cost F b → a < b :=
420   (fun h => match Nat.lt_or_ge a b with
421     | Or.inl hl => hl
422     | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left F.floor hge))),
423   fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos
424
425 theorem cost_mono (F : Frame) {a b : Nat} (h : a ≤ b) : cost F a ≤ cost F b :=
426   Nat.mul_le_mul_left F.floor h
427
428 /-- The ceiling: the energy order is the op order in every frame. -/
429 theorem frame_independent (F₁ F₂ : Frame) (g v : Nat → Nat) :
430   (∀ N, cost F₁ (v N) < cost F₁ (g N)) → (∀ N, cost F₂ (v N) < cost F₂ (g N)) :=
431   (fun h N => (cost_lt_iff F₂ _).mpr ((cost_lt_iff F₁ _).mp (h N))),
432   fun h N => (cost_lt_iff F₁ _).mpr ((cost_lt_iff F₂ _).mp (h N)))
433
434 /-- The Arrhenius frame of the April paper, at its scope. -/
435 structure Relax where
436   barrier : Nat → Nat
437   grows : ∀ N, N ≤ barrier N
438   escape : Nat → Nat
439   arrhenius : ∀ N, 2 ^ barrier N ≤ escape N

```

```

440 theorem relaxation_exponential (R : Relax) (N : Nat) :  $2^N \leq R.escape\ N :=$ 
441   Nat.le_trans (Nat.pow_le_pow_right (by decide) (R.grows N)) (R.arrhenius N)
442
443 end KINETIC
444
445 /-! ## IV · The black-box bound, pigeonhole without axioms -/
446 namespace BLACKBOX
447
448 def has : List Nat → Nat → Bool
449 | [], _ => false
450 | y :: ys, k => Nat.beq y k || has ys k
451
452 def drop : List Nat → Nat → List Nat
453 | [], _ => []
454 | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)
455
456 theorem beq_self :  $\forall n : \text{Nat}, \text{Nat.beq } n\ n = \text{true}$ 
457 | 0 => rfl
458 | n + 1 => beq_self n
459
460 theorem drop_le :  $\forall (l : \text{List Nat}) (k : \text{Nat}), (\text{drop } l\ k).length \leq l.length$ 
461 | [], _ => Nat.le_refl 0
462 | y :: ys, k => by
463   unfold drop
464   cases Nat.beq y k with
465   | true => exact Nat.le_succ_of_le (drop_le ys k)
466   | false => exact Nat.succ_le_succ (drop_le ys k)
467
468 theorem drop_len :  $\forall (l : \text{List Nat}) (k : \text{Nat}), \text{has } l\ k = \text{true} \rightarrow (\text{drop } l\ k).length + 1 \leq l.length$ 
469 | [], _, h => Bool.noConfusion h
470 | y :: ys, k, h => by
471   unfold drop
472   unfold has at h
473   cases hy : Nat.beq y k with
474   | true => exact Nat.succ_le_succ (drop_le ys k)
475   | false =>
476     rw [hy] at h
477     exact Nat.succ_le_succ (drop_len ys k h)
478
479 theorem has_drop :  $\forall (l : \text{List Nat}) (k\ x : \text{Nat}), \text{Nat.beq } x\ k = \text{false} \rightarrow$ 
480   has  $l\ x = \text{true} \rightarrow$  has (drop  $l\ k$ )  $x = \text{true}$ 
481 | [], _, _, h => Bool.noConfusion h
482 | y :: ys, k, x, hxk, h => by
483   unfold drop
484   unfold has at h
485   cases hy : Nat.beq y k with
486   | true =>
487     cases hyx : Nat.beq y x with
488     | true =>
489       have :  $x = k := (\text{Nat.eq_of_beq_eq_true } hyx).symm.trans$ 
490         (Nat.eq_of_beq_eq_true hy)
491       rw [this, beq_self] at hxk; exact Bool.noConfusion hxk
492     | false =>
493       rw [hyx] at h
494       exact has_drop ys k x hxk h
495   | false =>
496     show has (y :: drop ys k)  $x = \text{true}$ 
497     unfold has
498     cases hyx : Nat.beq y x with
499     | true => rfl
500     | false =>
501       rw [hyx] at h
502       exact has_drop ys k x hxk h
503
504 theorem beq_false_of_lt (x K : Nat) (h :  $x < K$ ) : Nat.beq x K = false := by
505   cases hb : Nat.beq x K with
506   | false => rfl
507   | true =>
508     have e :  $x = K := \text{Nat.eq_of_beq_eq_true } hb$ 
509     rw [e] at h; exact absurd h (Nat.lt_irrefl K)
510
511 theorem pigeon :  $\forall (K : \text{Nat}) (l : \text{List Nat}), (\forall x, x < K \rightarrow \text{has } l\ x = \text{true}) \rightarrow K \leq l.length$ 
512 | 0, _, _ => Nat.zero_le _
513 | K + 1, l, h =>
514   Nat.le_trans
515   (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
516     has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx))))))
517   (drop_len l K (h K (Nat.lt_succ_self K)))
518
519 /-! The run of a test-only procedure on the all-false path: its queries form a
520 fixed list, adaptivity collapsing because every answer is false. -/
521 def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
522
523 theorem run_eq_has :  $\forall (l : \text{List Nat}) (x : \text{Nat}), \text{run } l\ (\text{fun } y => \text{Nat.beq } y\ x) = \text{has } l\ x$ 
524 | [], _ => rfl
525 | y :: ys, x => by
526   show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
527   rw [run_eq_has ys x]
528
529 /-! The black-box bound. A test-only procedure correct for  $\exists$  over K candidates
530 tests at least K of them on the all-false path. -/
531 theorem black_box_bound (K : Nat) (path : List Nat)
532   (correct :  $\forall f : \text{Nat} \rightarrow \text{Bool}, \text{run path } f = \text{true} \rightarrow \exists x, x < K \wedge f\ x = \text{true}$ ) :
533    $K \leq \text{path.length} :=$ 
534   pigeon K path (fun x hx => by
535     rw [run_eq_has]
536     exact (correct _).mpr (x, hx, beq_self x))
537
538 end BLACKBOX
539
540 /-! ## V · The asymmetry chain: kinetic plus block -/
541 namespace ASYM
542 open KINETIC
543
544 /-! Link 1 to 4. Over  $2^N$  candidates a test-only generator queries at least
545  $2^N$  times; each query pays the floor; verification of one witness of v ops
546 costs less whenever  $v < 2^N$ ; and at N past the budget the generator exhausts it. -/
547 theorem asymmetry_chain (F : Frame) (N v : Nat) (hv :  $v < 2^N$ ) (path : List Nat)
548   (correct :  $\forall f : \text{Nat} \rightarrow \text{Bool}, \text{BLACKBOX.run path } f = \text{true} \rightarrow \exists x, x < 2^N \wedge f\ x = \text{true}$ ) :
549    $2^N \leq \text{path.length} \wedge$ 
550    $\text{cost } F\ v < \text{cost } F\ (2^N) \wedge$ 
551    $\text{cost } F\ (2^N) \leq \text{cost } F\ \text{path.length} \wedge$ 
552    $(F.\text{budget} \leq N \rightarrow F.\text{budget} < \text{cost } F\ \text{path.length}) :=$  by
553   have hb :  $2^N \leq \text{path.length} := \text{BLACKBOX.black\_box\_bound } (2^N) \text{ path correct}$ 
554   refine (hb, (cost_lt_iff F v (2^N)).mpr hv, cost_mono F hb, fun hN => ?_)
555   have h1 :  $F.\text{budget} < 2^N \wedge F.\text{budget} := \text{Nat.lt\_two\_pow\_self}$ 
556   have h2 :  $2^N \wedge F.\text{budget} \leq 2^N := \text{Nat.pow\_le\_pow\_right (by decide) hN}$ 
557   have h3 :  $2^N \leq \text{cost } F\ (2^N) := \text{Nat.le\_mul\_of\_pos\_left } (2^N) F.\text{floor\_pos}$ 
558   exact Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le h1 h2) h3)
559   (cost_mono F hb)
560
561 /-! Link 5. The asymmetry is real at its scope and decides nothing beyond it:
562 the energy gap equals the op gap in every frame (A3 kinetic face), and a record
563 shared by both worlds returns no value (A3 block face). -/
564 theorem asymmetry_decides_nothing (F1 F2 : Frame) (g v : Nat → Nat) :
565    $(\forall N, \text{cost } F_1\ (v\ N) < \text{cost } F_1\ (g\ N)) \rightarrow (\forall N, \text{cost } F_2\ (v\ N) < \text{cost } F_2\ (g\ N)) \wedge$ 
566    $(\neg \exists r : \text{Nat} \rightarrow \text{Bool}, \forall w : \text{Nat} \times \text{Bool},$ 
567      $(w = \text{BLOCKS.equalWorld } v\ w = \text{BLOCKS.separatedWorld}) \rightarrow r\ w.1 = w.2) :=$ 
568   (frame_independent F1 F2 g v, BLOCKS.block_chain.2.2.2.1)
569
570 /-! The asymmetry chain, terminated. The blocks hold, the black-box generator
571 pays exponentially at the floor and exhausts any budget, verification is
572 cheaper, the kinetic gap is the op gap, the shared record decides nothing,
573 the seat is vacant, the throne is empty, and the emission is [.]. -/
574 theorem asymmetry_terminal (F : Frame) (N v : Nat) (hv :  $v < 2^N$ ) (path : List
575   Nat)
576   (correct :  $\forall f : \text{Nat} \rightarrow \text{Bool}, \text{BLACKBOX.run path } f = \text{true} \rightarrow \exists x, x < 2^N \wedge f\ x = \text{true}$ ) :
577    $(2^N \leq \text{path.length} \wedge \text{cost } F\ v < \text{cost } F\ (2^N) \wedge$ 
578      $\text{cost } F\ (2^N) \leq \text{cost } F\ \text{path.length} \wedge (F.\text{budget} \leq N \rightarrow F.\text{budget} < \text{cost } F\ \text{path.length})) \wedge$ 
579    $(\forall F' : \text{Frame}, \forall g\ w : \text{Nat} \rightarrow \text{Nat},$ 
580      $(\forall n, \text{cost } F\ (w\ n) < \text{cost } F\ (g\ n)) \rightarrow (\forall n, \text{cost } F'\ (w\ n) < \text{cost } F'\ (g\ n))) \wedge$ 
581    $(\forall l : \text{List Bool}, \text{THRONE.allFixed } l = \text{true} \rightarrow l = []) \wedge$ 
582    $(\neg \forall RA : \text{Prop}, RA) \wedge$ 
583    $\text{THRONE.chainVerdict} = \text{.dot} :=$ 
584   (asymmetry_chain F N v hv path correct,
585     fun F' g w => frame_independent F F' g w,
586     THRONE.seat_vacant, THRONE.throne_empty, rfl)
587
588 /-! The hypothesis is inhabited: exhaustive search over two candidates is a
589 correct test-only procedure, so the chain is not vacuous. -/
590 theorem exhaustive_correct_two (f : Nat → Bool) :
591   BLACKBOX.run [0, 1] f = true →  $\exists x, x < 2^1 \wedge f\ x = \text{true} :=$  by
592   constructor
593   · intro h
594     show  $\exists x, x < 2^1 \wedge f\ x = \text{true}$ 
595     cases h0 : f 0 with
596     | true => exact (0, by decide, h0)
597     | false =>
598       have h' : (f 0 || (f 1 || false)) = true := h
599       rw [h0] at h'
600       cases h1 : f 1 with
601       | true => exact (1, by decide, h1)
602       | false => rw [h1] at h'; exact Bool.noConfusion h'
603   · intro (x, hx, hf)
604     show (f 0 || (f 1 || false)) = true
605     match x, hx, hf with
606     | 0, _, hf => rw [hf]; rfl
607     | 1, _, hf => rw [hf]; cases f 0 <=> rfl
608     | n + 2, hx, _ => exact absurd (Nat.lt_of_le_of_lt (Nat.le_add_left 2 n) hx)
609     (Nat.lt_irrefl 2)
610
611 /-! The chain instantiated at N = 1 on the unit frame. -/
612 def unitFrame : Frame := (1, by decide, 1)
613
614 theorem asymmetry_instance :
615    $2^1 \leq [0, 1].length \wedge \text{cost unitFrame } 1 < \text{cost unitFrame } (2^1) :=$ 
616   (asymmetry_chain unitFrame 1 1 (by decide) [0, 1] exhaustive_correct_two).1,
617   (asymmetry_chain unitFrame 1 1 (by decide) [0, 1] exhaustive_correct_two).2.1)
618
619 end ASYM
620 end PNP.CHAIN
621
622 /-! ## Cones, pinned: every theorem axiom-free -/
623 /-! info: 'PNP.CHAIN.BLOCKS.a1_no_seat' does not depend on any axioms -/
624 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_seat
625 /-! info: 'PNP.CHAIN.BLOCKS.a1_no_carrier' does not depend on any axioms -/
626 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_carrier
627 /-! info: 'PNP.CHAIN.BLOCKS.a2_orientation_blind' does not depend on any axioms -/
628 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a2_orientation_blind
629 /-! info: 'PNP.CHAIN.BLOCKS.a3_record_decides_nothing' does not depend on any axioms -/
630 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a3_record_decides_nothing
631 /-! info: 'PNP.CHAIN.BLOCKS.a4_constant_reads_nothing' does not depend on any axioms -/
632 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a4_constant_reads_nothing
633 /-! info: 'PNP.CHAIN.BLOCKS.a5_rung_law' does not depend on any axioms -/
634 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a5_rung_law
635 /-! info: 'PNP.CHAIN.BLOCKS.a6_aperture' does not depend on any axioms -/
636 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a6_aperture
637 /-! info: 'PNP.CHAIN.BLOCKS.a7_reflexive_closure' does not depend on any axioms -/
638 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a7_reflexive_closure
639 /-! info: 'PNP.CHAIN.BLOCKS.a8_soundness_cell' does not depend on any axioms -/
640 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a8_soundness_cell
641 /-! info: 'PNP.CHAIN.BLOCKS.block_chain' does not depend on any axioms -/
642 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.block_chain
643 /-! info: 'PNP.CHAIN.BLOCKS.seat_vacant' does not depend on any axioms -/
644 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.seat_vacant
645 /-! info: 'PNP.CHAIN.BLOCKS.throne_empty' does not depend on any axioms -/
646 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.throne_empty

```

```

642 #guard_msgs in #print axioms PNP.CHAIN.THRONE.throne_empty
643 /-- info: 'PNP.CHAIN.THRONE.root_forces_no_value' does not depend on any axioms -/
644 #guard_msgs in #print axioms PNP.CHAIN.THRONE.root_forces_no_value
645 /-- info: 'PNP.CHAIN.THRONE.dot_is_final' does not depend on any axioms -/
646 #guard_msgs in #print axioms PNP.CHAIN.THRONE.dot_is_final
647 /-- info: 'PNP.CHAIN.THRONE.block_chain_terminal' does not depend on any axioms -/
648 #guard_msgs in #print axioms PNP.CHAIN.THRONE.block_chain_terminal
649 /-- info: 'PNP.CHAIN.KINETIC.no_free_registration' does not depend on any axioms -/
650 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.no_free_registration
651 /-- info: 'PNP.CHAIN.KINETIC.cost_lt_iff' does not depend on any axioms -/
652 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_lt_iff
653 /-- info: 'PNP.CHAIN.KINETIC.cost_mono' does not depend on any axioms -/
654 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_mono
655 /-- info: 'PNP.CHAIN.KINETIC.frame_independent' does not depend on any axioms -/
656 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.frame_independent
657 /-- info: 'PNP.CHAIN.KINETIC.relaxation_exponential' does not depend on any axioms
658 -/
659 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.relaxation_exponential
660 /-- info: 'PNP.CHAIN.BLACKBOX.beq_self' does not depend on any axioms -/
661 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_self
662 /-- info: 'PNP.CHAIN.BLACKBOX.drop_le' does not depend on any axioms -/
663 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_le
664 /-- info: 'PNP.CHAIN.BLACKBOX.drop_len' does not depend on any axioms -/
665 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_len
666 /-- info: 'PNP.CHAIN.BLACKBOX.has_drop' does not depend on any axioms -/
667 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.has_drop
668 /-- info: 'PNP.CHAIN.BLACKBOX.beq_false_of_lt' does not depend on any axioms -/
669 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_false_of_lt
670 /-- info: 'PNP.CHAIN.BLACKBOX.pigeon' does not depend on any axioms -/
671 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.pigeon
672 /-- info: 'PNP.CHAIN.BLACKBOX.run_eq_has' does not depend on any axioms -/
673 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.run_eq_has
674 /-- info: 'PNP.CHAIN.BLACKBOX.black_box_bound' does not depend on any axioms -/
675 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.black_box_bound
676 /-- info: 'PNP.CHAIN.ASYM.asymmetry_chain' does not depend on any axioms -/
677 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_chain
678 /-- info: 'PNP.CHAIN.ASYM.asymmetry_decides_nothing' does not depend on any axioms
679 -/
680 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_decides_nothing
681 /-- info: 'PNP.CHAIN.ASYM.asymmetry_terminal' does not depend on any axioms -/
682 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_terminal
683 /-- info: 'PNP.CHAIN.ASYM.exhaustive_correct_two' does not depend on any axioms -/
684 #guard_msgs in #print axioms PNP.CHAIN.ASYM.exhaustive_correct_two
685 /-- info: 'PNP.CHAIN.ASYM.asymmetry_instance' does not depend on any axioms -/
686 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_instance
687 -/
688 /--!
689 # PNP.VANISH · the vanishing GOL of [.] on the RA-RAM locus bridge
690
691 The bridge (Bridge_Final.lean) joins the kinetic register, RA, the deed, to the
692 formal
693 register, RAM, the reading, at one locus: the fixed set of the native fold. Its
694 carrier
695 halts exactly when the line property holds, and its socket is exactly that property.
696 It carries a supplied bit to a seat and manufactures none.
697
698 On the Riemann stage the fold is  $s \mapsto 1 - s$ ; the locus is the line, inhabited, Ground
699 dimension one: the bridge carries one keyed bit to a seat.
700
701 On the P versus NP stage the fold is complementation,  $b \mapsto \neg b$ . The locus vanishes:
702 Ground dimension zero. The bridge is unchanged and its socket collapses onto the
703 vacancy: the carrier halts iff the zero set is empty. That is the vanishing GOL.
704 Three faces lock on one object, and the object is the empty seat:
705
706 L · the Tongue: a vanished seat emits [.] , outside the economy, owing nothing;
707 G · the Form: complementation has no fixed point, Ground dimension zero, counted;
708 M · the Number: least erasure holds of the empty configuration and of it alone.
709
710 The kinetic face reads the same lock: silence registers no bit and pays no floor,
711 while any emission of a value registers a bit and pays it. The denial asymmetry
712 sorts the two locks that meet here: the vacancy is keyless, its denial incoherent
713 on every frame of the class; the separation is keyed, its denial a coherent world,
714 and both worlds present the bridge the same vanished frame, so the lock that forms
715 determines the vacancy totally and the value not at all.
716
717 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
718 -/
719
720 namespace PNP.VANISH
721
722 /--! ## 0 · tokens -/
723 inductive Token : Type
724 | seal | broken | opn | dot
725 deriving DecidableEq, Repr
726
727 def Token.inEconomy : Token → Bool
728 | .dot => false
729 | _ => true
730
731 /--! ## I · the bridge, carried from Bridge_Final.lean, axiom-free -/
732 structure Frame where
733   S : Type
734   τ : S → S
735   Z : S → Prop
736   involutive : ∀ s, τ (τ s) = s
737   symmetric : ∀ s, Z s → Z (τ s)
738
739 def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
740
741 inductive Tri : Type
742 | tt | ff | bot
743 deriving DecidableEq, Repr
744
745 structure Carrier (X : Frame) where
746   terminal : Tri
747   shadow : terminal = .bot → LineProperty X
748
749

```

```

745 theorem halted_iff (X : Frame) : (∃ b : Carrier X, b.terminal = .bot) → LineProperty
746 X :=
747   (fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _ => rfl)), rfl))
748
749 class Supplied (X : Frame) where
750   term : LineProperty X
751
752 theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) → LineProperty X
753 :=
754   (fun (i) => i.term, fun t => {(t)})
755
756 /--! ## II · the two stages: the locus inhabited, the locus vanished -/
757
758 /-- The Riemann fold in the doubled real coordinate. -/
759 def tR (p : Int × Int) : Int × Int := (2 - p.1, p.2)
760
761 /-- The Riemann locus is inhabited: the point h = 1 is fixed. -/
762 theorem rh_locus_inhabited : tR (1, 0) = (1, 0) := by decide
763
764 /-- Ground dimension one on the Riemann window  $h \in \{0, 1, 2\}$ , counted. -/
765 theorem rh_ground_one : (([0, 1, 2] : List Int).filter (fun h => 2 - h == h)).length
766 = 1 := by
767   decide
768
769 /-- The P versus NP fold. -/
770 def comp (b : Bool) : Bool := !b
771
772 theorem comp_involutive : ∀ b, comp (comp b) = b
773 | true => rfl
774 | false => rfl
775
776 /-- G · the locus vanishes. -/
777 theorem locus_vanishes : ¬ ∃ b, comp b = b
778 | (true, h) => Bool.noConfusion h
779 | (false, h) => Bool.noConfusion h
780
781 def groundDim : Nat := ([false, true].filter (fun b => comp b == b)).length
782
783 /-- G · Ground dimension zero, counted. -/
784 theorem ground_dim_zero : groundDim = 0 := rfl
785
786 /-- The P versus NP frame, for any fold-symmetric zero set. -/
787 def cframe (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) : Frame :=
788   (Bool, comp, Z, comp_involutive, hZ)
789
790 /--! ## III · the bridge on the vanished locus -/
791
792 /-- The line property collapses onto the vacancy of the zero set. -/
793 theorem line_iff_empty (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) :
794   LineProperty (cframe Z hZ) → ∀ b, ¬ Z b :=
795     (fun h b hb => locus_vanishes (b, h b hb), fun h b hb => absurd hb (h b))
796
797 /-- The carrier halts exactly when nothing stands to be carried. -/
798 theorem carrier_halts_iff_empty (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) :
799   (∃ c : Carrier (cframe Z hZ), c.terminal = .bot) → ∀ b, ¬ Z b :=
800     (halted_iff _).trans (line_iff_empty Z hZ)
801
802 /-- The socket vanishes with the locus. -/
803 theorem socket_vanishes (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) :
804   Nonempty (Supplied (cframe Z hZ)) → ∀ b, ¬ Z b :=
805     (socket_is_the_property _).trans (line_iff_empty Z hZ)
806
807 /-- A charged configuration has no socket: nothing can be supplied onto an empty
808 seat. -/
809 theorem no_socket_when_charged :
810   ¬ Nonempty (Supplied (cframe (fun _ => True) (fun _ => trivial))) :=
811     fun h => (socket_vanishes _).mp h true trivial
812
813 /-- The empty configuration halts the carrier with nothing carried. -/
814 theorem empty_halts :
815   ∃ c : Carrier (cframe (fun _ => False) (fun _ h => h)), c.terminal = .bot :=
816     (carrier_halts_iff_empty _).mpr (fun _ h => h)
817
818 /--! ## IV · the three faces of the vanishing GOL -/
819
820 /-- L · the seat token: a vanished seat emits the dot; an inhabited seat with its
821 bit unsupplied stays open. -/
822 def seatToken : Nat → Token
823 | 0 => .dot
824 | _ + 1 => .opn
825
826 theorem tongue_face :
827   seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy = false ∧ seatToken
828   1 = .opn :=
829     (rfl, rfl, rfl)
830
831 def allFixed : List Bool → Bool
832 | [] => true
833 | b :: bs => (comp b == b) && allFixed bs
834
835 /-- M · least erasure determines exactly one configuration, the empty one. -/
836 theorem number_face : (∀ l, allFixed l = true → l = []) ∧ allFixed [] = true :=
837   (fun l => match l with
838     | [] => fun _ => rfl
839     | true :: _ => fun h => Bool.noConfusion h
840     | false :: _ => fun h => Bool.noConfusion h, rfl)
841
842 /--! ## V · the kinetic face: silence pays nothing, emission pays the floor -/
843 structure KFrame where
844   floor : Nat
845   floor_pos : 0 < floor
846
847 def cost (F : KFrame) (bits : Nat) : Nat := F.floor * bits
848
849 theorem kinetic_face (F : KFrame) : cost F 0 = 0 ∧ ∀ k, 0 < k → 0 < cost F k :=
850   (rfl, fun _ hk => Nat.mul_pos F.floor_pos hk)
851

```

```

847 /-! ## VI · the denial asymmetry at the vanished locus -/
848
849 /-- The vacancy is keyless: it holds on every frame of the class, and its denial,
850 a fixed point of complementation, is incoherent. -/
851 theorem vacancy_keyless :
852   (V (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) (s : (cframe Z hZ).S),
853     (cframe Z hZ).τ s ≠ s) ∧ ¬ ∃ b, comp b = b :=
854   (fun _ _ s h => locus_vanishes (s, h), locus_vanishes)
855
856 /-- The separation is keyed: both worlds are coherent, and both present the bridge
857 the same vanished frame, so no reading of what the bridge reads returns the value.
858 -/
859 def readOf (_sep : Bool) : Nat := groundDim
860
861 theorem separation_keyed : ¬ ∃ g : Nat → Bool, ∀ sep : Bool, g (readOf sep) = sep :=
862   fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))
863
864 /-! ## VII · the vanishing GOL, whole -/
865 theorem vanishing_gol :
866   -- the contrast: the Riemann seat is inhabited, Ground dimension one
867   (τR (1, 0) = (1, 0) ∧ (([0, 1, 2] : List Int).filter (fun h => 2 - h ==
868     h)).length = 1) ∧
869   -- G · the P versus NP seat vanishes, Ground dimension zero
870   ((¬ ∃ b, comp b = b) ∧ groundDim = 0) ∧
871   -- the bridge on the vanished seat: carrier and socket collapse onto the vacancy
872   (V (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)),
873     ((∃ c : Carrier (cframe Z hZ), c.terminal = .bot) → V b, ¬ Z b) ∧
874     (Nonempty (Supplied (cframe Z hZ)) → V b, ¬ Z b)) ∧
875   -- M · least erasure locks on the empty configuration alone
876   ((∀ l, allFixed l = true → l = []) ∧ allFixed [] = true) ∧
877   -- L · the vanished seat emits the dot, outside the economy
878   (seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy = false) ∧
879   -- the kinetic face: the dot pays nothing, any emission pays the floor
880   (V F : KFrame, cost F 0 = 0 ∧ ∀ k, 0 < k → 0 < cost F k) ∧
881   -- the denial asymmetry: the vacancy keyless, the value keyed and unread
882   (¬ ∃ b, comp b = b) ∧ (¬ ∃ g : Nat → Bool, V sep : Bool, g (readOf sep) = sep)
883   :=
884   ((rh_locus_inhabited, rh_ground_one), (locus_vanishes, ground_dim_zero),
885     fun Z hZ => (carrier_halts_iff_empty Z hZ, socket_vanishes Z hZ),
886     number_face, (tongue_face.1, tongue_face.2.1), kinetic_face,
887     vacancy_keyless.2, separation_keyed)
888
889 end PNP.VANISH
890
891 /-! ## Cones, pinned: every theorem axiom-free -/
892 /-- info: 'PNP.VANISH.halted_iff' does not depend on any axioms -/
893 #guard_msgs in #print axioms PNP.VANISH.halted_iff
894 /-- info: 'PNP.VANISH.socket_is_the_property' does not depend on any axioms -/
895 #guard_msgs in #print axioms PNP.VANISH.socket_is_the_property
896 /-- info: 'PNP.VANISH.rh_locus_inhabited' does not depend on any axioms -/
897 #guard_msgs in #print axioms PNP.VANISH.rh_locus_inhabited
898 /-- info: 'PNP.VANISH.rh_ground_one' does not depend on any axioms -/
899 #guard_msgs in #print axioms PNP.VANISH.rh_ground_one
900 /-- info: 'PNP.VANISH.comp_involutive' does not depend on any axioms -/
901 #guard_msgs in #print axioms PNP.VANISH.comp_involutive
902 /-- info: 'PNP.VANISH.locus_vanishes' does not depend on any axioms -/
903 #guard_msgs in #print axioms PNP.VANISH.locus_vanishes
904 /-- info: 'PNP.VANISH.ground_dim_zero' does not depend on any axioms -/
905 #guard_msgs in #print axioms PNP.VANISH.ground_dim_zero
906 /-- info: 'PNP.VANISH.line_iff_empty' does not depend on any axioms -/
907 #guard_msgs in #print axioms PNP.VANISH.line_iff_empty
908 /-- info: 'PNP.VANISH.carrier_halts_iff_empty' does not depend on any axioms -/
909 #guard_msgs in #print axioms PNP.VANISH.carrier_halts_iff_empty
910 /-- info: 'PNP.VANISH.socket_vanishes' does not depend on any axioms -/
911 #guard_msgs in #print axioms PNP.VANISH.socket_vanishes
912 /-- info: 'PNP.VANISH.no_socket_when_charged' does not depend on any axioms -/
913 #guard_msgs in #print axioms PNP.VANISH.no_socket_when_charged
914 /-- info: 'PNP.VANISH.empty_halts' does not depend on any axioms -/
915 #guard_msgs in #print axioms PNP.VANISH.empty_halts
916 /-- info: 'PNP.VANISH.tongue_face' does not depend on any axioms -/
917 #guard_msgs in #print axioms PNP.VANISH.tongue_face
918 /-- info: 'PNP.VANISH.number_face' does not depend on any axioms -/
919 #guard_msgs in #print axioms PNP.VANISH.number_face
920 /-- info: 'PNP.VANISH.kinetic_face' does not depend on any axioms -/
921 #guard_msgs in #print axioms PNP.VANISH.kinetic_face
922 /-- info: 'PNP.VANISH.vacancy_keyless' does not depend on any axioms -/
923 #guard_msgs in #print axioms PNP.VANISH.vacancy_keyless
924 /-- info: 'PNP.VANISH.separation_keyed' does not depend on any axioms -/
925 #guard_msgs in #print axioms PNP.VANISH.separation_keyed
926 /-- info: 'PNP.VANISH.vanishing_gol' does not depend on any axioms -/
927 #guard_msgs in #print axioms PNP.VANISH.vanishing_gol
928
929 /-!
930 # PNP.MISSING · the missing object, located
931
932 The asymmetry chain seals a test-only bound. Its adversary family is the point
933 predicate y ↦ (y = x): a tester that sees only answers must query every candidate.
934 Read as descriptions, the same family is solved in one step: the description of
935 a point is the point. The black-box hardness lives entirely in the hiding.
936
937 So the missing object is a hardness statement over descriptions: a family of
938 instances, given whole to the solver, that no polynomial-time reader of the
939 description decides. That is the white-box lower bound, P ≠ NP itself. It is a
940 fact about the model of computation, keyed: it holds in one coherent model and
941 fails in another, which is why nothing world-independent can supply it.
942 Core Lean 4, no import, no axiom of any kind.
943 -/
944
945 namespace PNP.MISSING
946
947 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
948   | 0 => rfl
949   | n + 1 => beq_self n
950
951 /-- The black-box adversary family of 'black_box_bound'. -/
952 def point (x : Nat) : Nat → Bool := fun y => Nat.beq y x

```

```

951 /-- The same family, handed over as descriptions. -/
952 structure Desc where
953   secret : Nat
954
955 def evalD (d : Desc) : Nat → Bool := point d.secret
956
957 /-- The white-box reader: one step, read the description. -/
958 def reader (d : Desc) : Nat := d.secret
959 def readerTime (_ : Nat) : Nat := 1
960
961 /-- The black-box hard instances are white-box trivial. -/
962 theorem reader_solves : ∀ d : Desc, evalD d (reader d) = true :=
963   fun d => beq_self d.secret
964
965 /-! ## the missing object, typed -/
966
967 structure Model where
968   Alg : Type
969   decides : Alg → Prop
970   time : Alg → Nat → Nat
971
972 def PolyBounded (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c
973
974 /-- THE MISSING OBJECT. Every algorithm that decides the problem from its
975 description runs in super-polynomial time. -/
976 def WhiteBoxBound (M : Model) : Prop :=
977   ∀ A : M.Alg, M.decides A → ~ PolyBounded (M.time A)
978
979 theorem reader_poly : PolyBounded readerTime :=
980   (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))
981
982 /-- A coherent model where descriptions reveal the witness: the object fails. -/
983 def revealing : Model := (Unit, fun _ => ∀ d : Desc, evalD d (reader d) = true, fun
984   _ => readerTime)
985
986 theorem fails_when_descriptions_reveal : ~ WhiteBoxBound revealing :=
987   fun h => h () reader_solves reader_poly
988
989 /-- The object is contingent on the model: it fails where descriptions reveal, and
990 holds
991 wherever no algorithm decides. No fact
992 true in both, kinetic, black-box or vanishing, can entail it. -/
993 theorem missing_object_is_keyed :
994   (∃ M : Model, ~ WhiteBoxBound M) ∧ (∀ M : Model, (∀ A, ~ M.decides A) →
995     WhiteBoxBound M) :=
996   ((revealing, fails_when_descriptions_reveal), fun _ h hA => absurd hA (h A))
997
998 end PNP.MISSING
999
1000 /-- info: 'PNP.MISSING.reader_solves' does not depend on any axioms -/
1001 #guard_msgs in #print axioms PNP.MISSING.reader_solves
1002 /-- info: 'PNP.MISSING.reader_poly' does not depend on any axioms -/
1003 #guard_msgs in #print axioms PNP.MISSING.reader_poly
1004 /-- info: 'PNP.MISSING.fails_when_descriptions_reveal' does not depend on any axioms -/
1005 #guard_msgs in #print axioms PNP.MISSING.fails_when_descriptions_reveal
1006 /-- info: 'PNP.MISSING.missing_object_is_keyed' does not depend on any axioms -/
1007 #guard_msgs in #print axioms PNP.MISSING.missing_object_is_keyed
1008
1009 /-!
1010 # PNP.PRIMEBRIDGE · the root freedom cut and prime-as-freedom against the missing
1011 object
1012
1013 Source of the method: Phys05† v1.0.5p, namespace PrimeFreedom. Freedom is the fibre
1014 of
1015 an invariant map; every prime is one fibre of exactly two, (1, p) and (p, 1), off
1016 the seat
1017 (prime_fibre, prime_off_seat, freedom_is_exactly_two).
1018
1019 The attack. Carry the cut to the white-box object. Over a semiprime n = p·q the
1020 multiplicative fibre has four points and two orbits under the swap fold: the trivial
1021 orbit {(1, n), (n, 1)} and the witness orbit {(p, q), (q, p)}. The description n is
1022 even under the fold. Two findings follow, both at theorem grade.
1023
1024 F1 · The freedom on the fibre is one orientation bit, and it is gauge: the reader
1025 (a, b) ↦ (min, max) is even, selects one point per orbit, and removes it.
1026 F2 · The description determines the witness orbit outright: informational freedom
1027 over the witness is zero. The missing object is therefore not a fibre fact.
1028 It is a cost fact: n hides nothing and still may resist every fast reader.
1029
1030 The new bridge. Prime freedom, read as cost, is the posit "no polynomial reader
1031 selects a nontrivial point of the semiprime fibre". Under the collapse field (P = NP
1032 gives polynomial search for NP relations, the classical search-to-decision theorem,
1033 carried as a field), the posit yields P ≠ NP: prime_bridge. The posit is keyed: a
1034 coherent machine refutes it (the Shor face). And it is strictly stronger than the
1035 separation: a coherent machine has the separation and an easy fibre, so the prime
1036 bridge is sufficient and never equivalent. Factoring lies in NP n coNP, so no
1037 equivalence is available unless NP = coNP.
1038
1039 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1040 -/
1041
1042 namespace PNP.PRIMEBRIDGE
1043
1044 /-! ## I · the semiprime fibre, executed at n = 15 -/
1045
1046 def mul_orbit (n a b : Nat) : Prop := a * b = n
1047
1048 def fibre (n : Nat) : List (Nat × Nat) :=
1049   (List.range (n + 1)).flatMap fun a =>
1050     (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none
1051
1052 def swap (p : Nat × Nat) : Nat × Nat := (p.2, p.1)
1053
1054 /-- The fibre over 15: four points. -/
1055 theorem fibre_15 : fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] := by decide

```

```

1051 -- Off the seat: no point of the fibre is fixed by the swap. -/
1052 theorem off_seat_15 : (fibre 15).all (fun p => swap p != p) = true := by decide
1053
1054 -- The fibre is swap-closed: two orbits, each of exactly two. -/
1055 theorem swap_closed_15 : (fibre 15).all (fun p => (fibre 15).contains (swap p)) =
1056   true := by decide
1057
1058 def trivialPt (p : Nat × Nat) : Bool := p.1 == 1 || p.2 == 1
1059
1060 -- The witness orbit: exactly two nontrivial points, one the swap of the other. -/
1061 theorem witness_orbit_15 :
1062   (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)] ∧ swap (3, 5) = (5,
1063     3) := by
1064     decide
1065
1066 --! ## II · F1 · the orientation bit is free and it is gauge -/
1067
1068 -- The wall on the orientation: the description is even, the order is odd, so no
1069 -- reading of the description returns the first factor on the witness orbit. -/
1070 theorem orientation_wall : ¬ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 * 3) = 5 :=
1071   fun (_, h1, h2) => absurd (h1.symm.trans h2) (by decide)
1072
1073 def canon (p : Nat × Nat) : Nat × Nat := if p.1 ≤ p.2 then p else swap p
1074
1075 -- The canonical reader is even under the fold. -/
1076 theorem canon_even : ∀ p ∈ fibre 15, canon (swap p) = canon p := by decide
1077
1078 -- F1 · the canonical reader takes each orbit to one point: the bit is gauge. -/
1079 theorem orientation_is_gauge :
1080   ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5), (3, 5)] := by
1081     decide
1082
1083 --! ## III · F2 · informational freedom over the witness is zero -/
1084
1085 -- The witness set is a function of the description, on every relation. -/
1086 theorem description_determines_witnesses {D W : Type} (R : D → W → Prop) (d₁ d₂ : D)
1087   (h : d₁ = d₂) : R d₁ = R d₂ := by rw [h]
1088
1089 -- F2 · at 15 the description fixes the canonical witness uniquely. -/
1090 theorem witness_determined_15 :
1091   (((fibre 15).filter (fun p => !trivialPt p)).map canon).eraseDups = [(3, 5)] :=
1092     by decide
1093
1094 --! ## IV · the cost model and the prime bridge -/
1095
1096 -- A machine: algorithms, their running times, and what each solves. -/
1097 structure Machine where
1098   Alg : Type
1099   time : Alg → Nat → Nat
1100   Solves : Alg → (Nat → Nat → Prop) → Prop
1101
1102 -- The factoring relation: p is a nontrivial divisor of n. -/
1103 def factorRel (n p : Nat) : Prop := 1 < p ∧ p < n ∧ ∃ q, p * q = n
1104
1105 def EasySearch (M : Machine) (R : Nat → Nat → Prop) : Prop :=
1106   ∃ A : M.Alg, M.Solves A R ∧ Poly (M.time A)
1107
1108 -- Prime freedom read as cost: no fast reader selects a point of the witness orbit.
1109 --/
1110 def PrimeHidden (M : Machine) : Prop := ¬ EasySearch M factorRel
1111
1112 -- The collapse field. 'PegNP' is the class equality on this machine; 'search' is the
1113 -- classical search-to-decision theorem specialized to factoring, an NP relation. -/
1114 structure Collapse (M : Machine) where
1115   PegNP : Prop
1116   search : PegNP → EasySearch M factorRel
1117
1118 -- THE PRIME BRIDGE. Hidden prime freedom yields the separation. -/
1119 theorem prime_bridge (M : Machine) (C : Collapse M) (hidden : PrimeHidden M) : ¬
1120   C.PegNP :=
1121     fun h => hidden (C.search h)
1122
1123 -- The Shor face: a coherent machine where the fibre is read fast. -/
1124 def revealing : Machine := (Unit, fun _ _ => 1, fun _ _ => True)
1125
1126 theorem one_is_poly : Poly (fun _ => 1) := (1, 0, fun n => Nat.le_add_left 1 (1 * n
1127   ^ 0))
1128
1129 -- The posit is keyed: it fails on a coherent machine. -/
1130 theorem prime_hidden_is_keyed : ¬ PrimeHidden revealing :=
1131   fun h => h ((), trivial, one_is_poly)
1132
1133 -- Strictly stronger than the separation: on the revealing machine with the class
1134 -- equality false, the separation holds and the posit fails. -/
1135 def separatedButRevealing : Collapse revealing := (False, fun h => absurd h id)
1136
1137 theorem prime_bridge_is_one_way :
1138   ¬ ∀ (M : Machine) (C : Collapse M), ¬ C.PegNP → PrimeHidden M :=
1139     fun h => prime_hidden_is_keyed (h revealing separatedButRevealing id)
1140
1141 --! ## V · the bridge, whole -/
1142 theorem the_prime_bridge :
1143   -- the fibre: four points, off the seat, swap-closed, one witness orbit
1144   (fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] ∧
1145     (fibre 15).all (fun p => swap p != p) = true ∧
1146     (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)]) ∧
1147   -- F1 · the orientation bit is walled and gauge
1148   ((¬ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 * 3) = 5) ∧
1149     ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5), (3, 5)]) ∧
1150   -- F2 · the description fixes the witness: zero informational freedom
1151   (((fibre 15).filter (fun p => !trivialPt p)).map canon).eraseDups = [(3, 5)] ∧
1152   -- the prime bridge, its key, and its direction
1153   (∀ (M : Machine) (C : Collapse M), PrimeHidden M → ¬ C.PegNP) ∧
1154   ¬ PrimeHidden revealing ∧

```

```

1155   (¬ ∀ (M : Machine) (C : Collapse M), ¬ C.PegNP → PrimeHidden M) :=
1156     ((fibre 15, off_seat_15, witness_orbit_15.1), (orientation_wall,
1157       orientation_is_gauge),
1158       witness_determined_15, prime_bridge, prime_hidden_is_keyed,
1159       prime_bridge_is_one_way)
1160
1161 end PNP.PRIMEBRIDGE
1162
1163 --! ## Cones, pinned: every theorem axiom-free -/
1164 -- info: 'PNP.PRIMEBRIDGE.fibre_15' does not depend on any axioms -/
1165 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.fibre_15
1166 -- info: 'PNP.PRIMEBRIDGE.off_seat_15' does not depend on any axioms -/
1167 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.off_seat_15
1168 -- info: 'PNP.PRIMEBRIDGE.swap_closed_15' does not depend on any axioms -/
1169 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.swap_closed_15
1170 -- info: 'PNP.PRIMEBRIDGE.witness_orbit_15' does not depend on any axioms -/
1171 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_orbit_15
1172 -- info: 'PNP.PRIMEBRIDGE.orientation_wall' does not depend on any axioms -/
1173 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_wall
1174 -- info: 'PNP.PRIMEBRIDGE.canon_even' does not depend on any axioms -/
1175 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.canon_even
1176 -- info: 'PNP.PRIMEBRIDGE.orientation_is_gauge' does not depend on any axioms -/
1177 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_is_gauge
1178 -- info: 'PNP.PRIMEBRIDGE.description_determines_witnesses' does not depend on any
1179   axioms -/
1180 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.description_determines_witnesses
1181 -- info: 'PNP.PRIMEBRIDGE.witness_determined_15' does not depend on any axioms -/
1182 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_determined_15
1183 -- info: 'PNP.PRIMEBRIDGE.prime_bridge' does not depend on any axioms -/
1184 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge
1185 -- info: 'PNP.PRIMEBRIDGE.one_is_poly' does not depend on any axioms -/
1186 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.one_is_poly
1187 -- info: 'PNP.PRIMEBRIDGE.prime_hidden_is_keyed' does not depend on any axioms -/
1188 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_hidden_is_keyed
1189 -- info: 'PNP.PRIMEBRIDGE.prime_bridge_is_one_way' does not depend on any axioms -/
1190 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge_is_one_way
1191 -- info: 'PNP.PRIMEBRIDGE.the_prime_bridge' does not depend on any axioms -/
1192 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.the_prime_bridge
1193
1194 --!
1195 # PNP.ROUND2 · the blocker of the prime bridge, located, and the second attack
1196
1197 ROUND 1 LEFT THE KEY ON 'PrimeHidden': no polynomial reader selects a point of the
1198 semiprime witness orbit. F2 proved the description carries the witness with zero
1199 informational freedom, so the key is a pure cost statement against readers that have
1200 everything. The blocker has three layers.
1201
1202 B1 · Relativization. A reader with a factoring oracle reads the orbit in one step.
1203   The posit fails relative to that oracle, so no argument valid relative to
1204   every
1205   oracle proves it (Baker-Gill-Solovay pattern).
1206 B2 · The self-gag. If the posit holds, factoring-based pseudorandom functions
1207   exist
1208   (Naor-Reingold), and then no natural property proves the circuit lower bound
1209   (Razborov-Rudich). The posit, if true, bars the natural route to itself.
1210 B3 · The floor points the wrong way. The black-box floor is proved unbreakable
1211   (no savings below 2^N for a test-only reader). Every known unconditional
1212   route
1213   to circuit lower bounds runs the other way: it needs savings below 2^N for
1214   white-box circuit SAT (Williams 2011, NEXP ≠ ACC*).
1215
1216 ROUND 2 attacks through B3, the one exit the barriers leave open: algorithms to
1217 lower
1218 bounds. The kinetic floor's black-box face cannot be beaten, by theorem here; the
1219 route requires beating its white-box face, which is exactly the access F2 located.
1220 The route reaches NEXP ≠ C. Two gaps separate it from P ≠ NP, and both are named.
1221
1222 Cited theorems enter as structure fields, never as 'axiom'. Core Lean 4, no import.
1223 Every theorem pinned at the foot: no axiom of any kind.
1224 --/
1225
1226 namespace PNP.ROUND2
1227
1228 --! ## I · the black-box floor admits no savings (axiom-free pigeonhole) -/
1229 namespace FLOOR
1230
1231 def has : List Nat → Nat → Bool
1232   | [], _ => false
1233   | y :: ys, k => Nat.beq y k || has ys k
1234
1235 def drop : List Nat → Nat → List Nat
1236   | [], _ => []
1237   | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)
1238
1239 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
1240   | 0 => rfl
1241   | n + 1 => beq_self n
1242
1243 theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
1244   | [], _ => Nat.le_refl 0
1245   | y :: ys, k => by
1246     unfold drop
1247     cases Nat.beq y k with
1248     | true => exact Nat.le_succ_of_le (drop_le ys k)
1249     | false => exact Nat.succ_le_succ (drop_le ys k)
1250
1251 theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length +
1252   1 ≤ l.length
1253   | [], _, h => Bool.noConfusion h
1254   | y :: ys, k, h => by
1255     unfold drop
1256     unfold has at h
1257     cases hy : Nat.beq y k with
1258     | true => exact Nat.succ_le_succ (drop_le ys k)
1259     | false =>
1260       rw [hy] at h
1261       exact Nat.succ_le_succ (drop_len ys k h)

```

```

1249
1250 theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
1251   has l x = true → has (drop l k) x = true
1252 | [], _, _, h => Bool.noConfusion h
1253 | y :: ys, k, x, hxx, h => by
1254   unfold drop
1255   unfold has at h
1256   cases hy : Nat.beq y k with
1257   | true =>
1258     cases hxx : Nat.beq y x with
1259     | true =>
1260       have : x = k := (Nat.eq_of_beq_eq_true hxx).symm.trans
1261         (Nat.eq_of_beq_eq_true hy)
1262       rw [this, beq_self] at hxx; exact Bool.noConfusion hxx
1263     | false =>
1264       rw [hxx] at h
1265       exact has_drop ys k x hxx h
1266   | false =>
1267     show has (y :: drop ys k) x = true
1268     unfold has
1269     cases hxx : Nat.beq y x with
1270     | true => rfl
1271     | false =>
1272       rw [hxx] at h
1273       exact has_drop ys k x hxx h
1274
1275 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
1276   cases hb : Nat.beq x K with
1277   | false => rfl
1278   | true =>
1279     have e : x = K := Nat.eq_of_beq_eq_true hb
1280     rw [e] at h; exact absurd h (Nat.lt_irrefl K)
1281
1282 theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤
1283   l.length
1284 | 0, _, _ => Nat.zero_le _
1285 | K + 1, l, h =>
1286   Nat.le_trans
1287   (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
1288     has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx))))))
1289   (drop_len l K (h K (Nat.lt_succ_self K)))
1290
1291 def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
1292
1293 theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has
1294   l x
1295 | [], _ => rfl
1296 | y :: ys, x => by
1297   show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
1298   rw [run_eq_has ys x]
1299
1300 /-- B3, the black-box face: no test-only reader saves a single query below 2^N. -/
1301 theorem black_box_no_savings (N : Nat) (path : List Nat)
1302   (correct : ∀ f : Nat → Bool, run path f = true ↔ ∃ x, x < 2 ^ N ∧ f x = true) :
1303   ~ path.length < 2 ^ N :=
1304   Nat.not_lt_of_le (pigeon (2 ^ N) path (fun x hx => by
1305     rw [~ run_eq_has]
1306     exact (correct _).mpr (x, hx, beq_self x)))
1307
1308 end FLOOR
1309
1310 /-- ## II · B1 · relativization, typed -/
1311
1312 /-- An oracle world: the posit read relative to an oracle. `factoring` is the oracle
1313   that answers the witness orbit; relative to it the posit fails. -/
1314 structure Relativized where
1315   Oracle      : Type
1316   Hidden      : Oracle → Prop
1317   factoring    : Oracle
1318   easy        : ~ Hidden factoring
1319
1320 /-- No argument valid relative to every oracle proves the posit. -/
1321 theorem b1_no_relativizing_proof (R : Relativized) : ~ ∀ 0, R.Hidden 0 :=
1322   fun h => R.easy (h R.factoring)
1323
1324 /-- ## III · B2 · the self-gag, typed -/
1325
1326 /-- The two cited links: hidden prime freedom gives pseudorandom functions
1327   (Naor-Reingold), and pseudorandom functions bar natural lower bounds
1328   (Razborov-Rudich). -/
1329 structure SelfGag where
1330   Hidden      : Prop
1331   PRF         : Prop
1332   NaturalLB   : Prop
1333   nr          : Hidden → PRF
1334   rr          : PRF → ~ NaturalLB
1335
1336 /-- If the posit holds, no natural argument proves the lower bound it needs. -/
1337 theorem b2_self_gag (G : SelfGag) (h : G.Hidden) : ~ G.NaturalLB :=
1338   G.rr (G.nr h)
1339
1340 /-- Equivalently: a natural proof of the lower bound would refute the posit. -/
1341 theorem b2_natural_refutes (G : SelfGag) (n : G.NaturalLB) : ~ G.Hidden :=
1342   fun h => b2_self_gag G h n
1343
1344 /-- ## IV · round 2 · algorithms to lower bounds, the open exit -/
1345
1346 /-- The Williams route, cited. `Savings C` is a white-box SAT algorithm for circuit
1347   class C running below 2^n by a superpolynomial factor. `williams` is the 2011
1348   theorem
1349   pattern: savings for C give NEXP ⊆ C. `scaleDown` and `generalize` are the two gaps
1350   to P ≠ NP: from NEXP down to NP, and from the restricted class C up to P/poly. -/
1351 structure Route where
1352   Class      : Type
1353   Savings    : Class → Prop
1354   NEXPnotIn  : Class → Prop
1355   williams    : ∀ C, Savings C → NEXPnotIn C

```

```

1351   Ppoly      : Class
1352   NNotPpoly  : Prop
1353   Sep        : Prop
1354   karp_sep    : NNotPpoly → Sep
1355
1356 /-- The route reaches NEXP ⊄ C on savings, and no further on its own fields. -/
1357 theorem round2_reaches (R : Route) (C : R.Class) (s : R.Savings C) : R.NEXPnotIn C
1358   :=
1359     R.williams C s
1360
1361 /-- What closes the route: savings for general circuits, plus the scale-down from
1362   NEXP to NP. Supplied both, the separation follows. -/
1363 theorem round2_closes (R : Route) (s : R.Savings R.Ppoly)
1364   (scaleDown : R.NEXPnotIn R.Ppoly → R.NNotPpoly) : R.Sep :=
1365     R.karp_sep (scaleDown (R.williams R.Ppoly s))
1366
1367 /-- The inversion, whole: the black-box floor cannot be beaten; the lower-bound
1368   route
1369   needs the white-box floor beaten; the posit gags the natural route; the posit fails
1370   relative to the factoring oracle. -/
1371 theorem round2_terminal :
1372   (∀ (N : Nat) (path : List Nat),
1373     (∀ f : Nat → Bool, FLOOR.run path f = true ↔ ∃ x, x < 2 ^ N ∧ f x = true) →
1374       ~ path.length < 2 ^ N) ∧
1375   (∀ R : Relativized, ~ ∀ 0, R.Hidden 0) ∧
1376   (∀ G : SelfGag, G.Hidden → ~ G.NaturalLB) ∧
1377   (∀ (R : Route) (C : R.Class), R.Savings C → R.NEXPnotIn C) ∧
1378   (∀ (R : Route), R.Savings R.Ppoly →
1379     (R.NEXPnotIn R.Ppoly → R.NNotPpoly) → R.Sep) :=
1380   (FLOOR.black_box_no_savings, b1_no_relativizing_proof, b2_self_gag,
1381     round2_reaches, round2_closes)
1382
1383 end PNP.ROUND2
1384
1385 /-- ## Cones, pinned: every theorem axiom-free -/
1386 /-- info: 'PNP.ROUND2.FLOOR.beq_self' does not depend on any axioms -/
1387 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_self
1388 /-- info: 'PNP.ROUND2.FLOOR.drop_le' does not depend on any axioms -/
1389 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_le
1390 /-- info: 'PNP.ROUND2.FLOOR.drop_len' does not depend on any axioms -/
1391 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_len
1392 /-- info: 'PNP.ROUND2.FLOOR.has_drop' does not depend on any axioms -/
1393 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.has_drop
1394 /-- info: 'PNP.ROUND2.FLOOR.beq_false_of_lt' does not depend on any axioms -/
1395 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_false_of_lt
1396 /-- info: 'PNP.ROUND2.FLOOR.pigeon' does not depend on any axioms -/
1397 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.pigeon
1398 /-- info: 'PNP.ROUND2.FLOOR.run_eq_has' does not depend on any axioms -/
1399 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.run_eq_has
1400 /-- info: 'PNP.ROUND2.FLOOR.black_box_no_savings' does not depend on any axioms -/
1401 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.black_box_no_savings
1402 /-- info: 'PNP.ROUND2.b1_no_relativizing_proof' does not depend on any axioms -/
1403 #guard_msgs in #print axioms PNP.ROUND2.b1_no_relativizing_proof
1404 /-- info: 'PNP.ROUND2.b2_self_gag' does not depend on any axioms -/
1405 #guard_msgs in #print axioms PNP.ROUND2.b2_self_gag
1406 /-- info: 'PNP.ROUND2.b2_natural_refutes' does not depend on any axioms -/
1407 #guard_msgs in #print axioms PNP.ROUND2.b2_natural_refutes
1408 /-- info: 'PNP.ROUND2.round2_reaches' does not depend on any axioms -/
1409 #guard_msgs in #print axioms PNP.ROUND2.round2_reaches
1410 /-- info: 'PNP.ROUND2.round2_closes' does not depend on any axioms -/
1411 #guard_msgs in #print axioms PNP.ROUND2.round2_closes
1412 /-- info: 'PNP.ROUND2.round2_terminal' does not depend on any axioms -/
1413 #guard_msgs in #print axioms PNP.ROUND2.round2_terminal
1414
1415 /--!
1416 # PNP.ROUTE · RA and Freedom as the only posits: where the route goes
1417
1418 Request: P ≠ NP with the Root Axiom and Freedom as the only posits, everything else
1419   axiom-free. This file decides whether that route exists.
1420
1421 I · RA is conservative. Its formal reading, ∀ x : U, 0 < ΔE x, is satisfied by
1422   interpreting U as an empty type. So any statement that does not mention U or
1423   ΔE
1424   and is derived from RA is derived without it.
1425 II · Freedom is conservative. Its formal reading, a fibre of exactly two distinct
1426   orientations, is satisfied by Bool. The same conclusion follows.
1427 III · Hence a derivation of the separation from RA and Freedom, as written, is a
1428   derivation of the separation from nothing. The route through the posits as
1429   written is the route without them.
1430 IV · The one reading of RA that carries the load: read on computation, it says
1431   every decider of SAT pays superpolynomially many registrations. That reading
1432   yields the separation in one step and is equivalent to it. It is the bridge,
1433   in RA's vocabulary.
1434
1435 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1436 -/
1437
1438 namespace PNP.ROUTE
1439
1440 /-- ## I · RA is conservative -/
1441
1442 /-- The Root Axiom in the codex's own formal reading (ROOT.RA). -/
1443 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1444
1445 /-- RA is satisfiable on any background: interpret U as the empty type. -/
1446 theorem ra_satisfiable : RA Empty (fun e => nomatch e) :=
1447   fun e => nomatch e
1448
1449 /-- RA is conservative: whatever follows from RA under every interpretation of its
1450   symbols, and does not mention them, holds outright. -/
1451 theorem ra_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → S) :
1452   S :=
1453     h Empty (fun e => nomatch e) ra_satisfiable
1454
1455 /-- ## II · Freedom is conservative -/
1456
1457 /-- Freedom in the codex's formal reading: a fibre of exactly two distinct
1458   orientations. -/

```

```

1454 structure Freedom where
1455   Fibre : Type
1456   a : Fibre
1457   b : Fibre
1458   distinct : a ≠ b
1459   exhaust : ∀ x : Fibre, x = a ∨ x = b
1460
1461 /-- Freedom is inhabited by the orientation bit itself. -/
1462 def boolFreedom : Freedom :=
1463   (Bool, true, false, Bool.noConfusion, fun x => match x with
1464     | true => Or.inl rfl
1465     | false => Or.inr rfl)
1466
1467 theorem freedom_conservative (S : Prop) (h : Freedom → S) : S := h boolFreedom
1468
1469 /-! ## III · the two posits together add nothing -/
1470
1471 /-- Any statement derived from RA and Freedom, uniformly in their interpretation,
1472 holds without them. -/
1473 theorem posits_add_nothing (S : Prop)
1474   (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → Freedom → S) : S :=
1475   h Empty (fun e => nomatch e) ra_satisfiable boolFreedom
1476
1477 /-! ## IV · the one reading that carries the load -/
1478
1479 def Poly (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c
1480
1481 /-- A machine model: algorithms, running time counted in registrations, and which
1482 algorithms decide SAT from its description. -/
1483 structure Machine where
1484   Alg : Type
1485   time : Alg → Nat → Nat
1486   decidesSAT : Alg → Prop
1487
1488 /-- The separation on a machine model: no polynomial-time decider of SAT. -/
1489 def Sep (M : Machine) : Prop := ¬ ∃ A : M.Alg, M.decidesSAT A ∧ Poly (M.time A)
1490
1491 /-- RA read on computation: every decider of SAT actuates superpolynomially many
1492 registrations. -/
1493 def RAComp (M : Machine) : Prop := ∀ A : M.Alg, M.decidesSAT A → ¬ Poly (M.time A)
1494
1495 /-- The separation, from RA read on computation. -/
1496 theorem sep_of_ra_comp (M : Machine) (h : RAComp M) : Sep M :=
1497   fun (A, hA, hP) => h A hA hP
1498
1499 /-- And conversely: the reading is the separation. -/
1500 theorem ra_comp_of_sep (M : Machine) (h : Sep M) : RAComp M :=
1501   fun A hA hP => h (A, hA, hP)
1502
1503 theorem ra_comp_is_sep (M : Machine) : RAComp M ↔ Sep M :=
1504   (sep_of_ra_comp M, ra_comp_of_sep M)
1505
1506 /-- RA read on computation is not conservative: it fails on a coherent machine. -/
1507 def fastMachine : Machine := (Unit, fun _ _ => 1, fun _ => True)
1508
1509 theorem ra_comp_is_keyed : ¬ RAComp fastMachine :=
1510   fun h => h () trivial (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))
1511
1512 /-! ## V · the route, whole -/
1513 theorem the_route :
1514   -- RA and Freedom as written add nothing to any statement outside their
1515   -- vocabulary
1516   (∀ S : Prop, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → Freedom → S) → S) ∧
1517   -- RA read on computation is exactly the separation
1518   (∀ M : Machine, RAComp M ↔ Sep M) ∧
1519   -- and that reading is keyed: a coherent machine refutes it
1520   ¬ RAComp fastMachine :=
1521   (posits_add_nothing, ra_comp_is_sep, ra_comp_is_keyed)
1522
1523 end PNP.ROUTE
1524
1525 /-! ## Cones, pinned: every theorem axiom-free -/
1526 /-- info: 'PNP.ROUTE.ra_satisfiable' does not depend on any axioms -/
1527 #guard_msgs in #print axioms PNP.ROUTE.ra_satisfiable
1528 /-- info: 'PNP.ROUTE.ra_conservative' does not depend on any axioms -/
1529 #guard_msgs in #print axioms PNP.ROUTE.ra_conservative
1530 /-- info: 'PNP.ROUTE.freedom_conservative' does not depend on any axioms -/
1531 #guard_msgs in #print axioms PNP.ROUTE.freedom_conservative
1532 /-- info: 'PNP.ROUTE.posits_add_nothing' does not depend on any axioms -/
1533 #guard_msgs in #print axioms PNP.ROUTE.posits_add_nothing
1534 /-- info: 'PNP.ROUTE.sep_of_ra_comp' does not depend on any axioms -/
1535 #guard_msgs in #print axioms PNP.ROUTE.sep_of_ra_comp
1536 /-- info: 'PNP.ROUTE.ra_comp_of_sep' does not depend on any axioms -/
1537 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_of_sep
1538 /-- info: 'PNP.ROUTE.ra_comp_is_sep' does not depend on any axioms -/
1539 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_sep
1540 /-- info: 'PNP.ROUTE.ra_comp_is_keyed' does not depend on any axioms -/
1541 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_keyed
1542 /-- info: 'PNP.ROUTE.the_route' does not depend on any axioms -/
1543 #guard_msgs in #print axioms PNP.ROUTE.the_route
1544
1545 /-!
1546 # PNP.THREESEAL · the Tongue and the Form added for direction, with RA beneath
1547
1548 The Number is orientation-blind by theorem. The architecture's answer is to read
1549 direction from the other two seals: Seal L, the Tongue, which carries order, and
1550 Seal G, the Form, which carries the fold. This file runs all three on P versus NP
1551 and asks the one question that decides the matter: does any seal read the direction
1552 from the problem, or is the direction supplied?
1553
1554 I · Each seal is computed on the problem. The problem is the same sentence and
1555 the same fold in the equal world and in the separated world, so each seal's
1556 reading is world-constant: the Tongue seals both sentences alike, the Form
1557 finds no fixed locus for either, the Number reads one magnitude for both.
1558
1559 II · No function of the three readings returns the orientation. Adding seals adds
1560 readings, and a coalition of world-constant readings is world-constant.
1561
1562 III · RA is keyless and conservative, so it selects no orientation.
1563
1564 IV · Given one supplied odd bit at the seat, the lock forms and locks exactly one
1565 orientation: the calibration is unique. This is the lock locking only one.
1566
1567 V · The verdict: sealed at the act, premise grade on the supplied bit; without
1568 the act, the dot.
1569
1570 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1571 -/
1572
1573 namespace PNP.THREESEAL
1574
1575 inductive Token : Type
1576 | sealSep | sealEq | broken | opn | dot
1577 deriving DecidableEq, Repr
1578
1579 /-- The two worlds, indexed by the separation bit. -/
1580 abbrev World := Bool
1581
1582 /-! ## I · the three seals, computed on the problem -/
1583
1584 /-- Seal L. A sentence is three slots of literal ids; the Tongue seals it when the
1585 three slots are populated and pairwise disjoint. It checks form, not truth. -/
1586 def disjointL : List Nat → List Nat → Bool
1587 | [], _ => true
1588 | x :: xs, ys => !(ys.contains x) && disjointL xs ys
1589
1590 def seall (s : List (List Nat)) : Bool :=
1591   match s with
1592   | [a, b, c] => !a.isEmpty && !b.isEmpty && !c.isEmpty &&
1593     disjointL a b && disjointL a c && disjointL b c
1594   | _ => false
1595
1596 /-- "P is not NP" and "P is NP": same existence slot, same relation slot, the
1597 kinetic slot differing only in the negation literal. -/
1598 def sentSep : List (List Nat) := [[101], [201, 209], [301]]
1599 def sentEq : List (List Nat) := [[101], [201], [301]]
1600
1601 theorem tongue_seals_both : seall sentSep = true ∧ seall sentEq = true := by decide
1602
1603 /-- Seal G. The fold is complementation; its fixed set, counted. -/
1604 def foldFixed : Nat := ([false, true].filter (fun b => (!b) == b)).length
1605
1606 theorem form_finds_no_locus : foldFixed = 0 := rfl
1607
1608 /-- Seal M. The lock scalar is the square of the signed scalar: reflection flips the
1609 sign and leaves the lock. -/
1610 def lockOf (signed : Int) : Int := signed * signed
1611
1612 theorem number_reads_one_magnitude : lockOf 1 = lockOf (-1) := by decide
1613
1614 /-- The three readings of the problem. The problem is fixed, so the readings do not
1615 depend on the world. -/
1616 def readings (w : World) : Bool × Nat × Int :=
1617   (seall sentSep && seall sentEq, foldFixed, lockOf 1)
1618
1619 theorem readings_world_constant : readings false = readings true := rfl
1620
1621 /-! ## II · no coalition of seals returns the orientation -/
1622 theorem seals_do_not_point :
1623   ¬ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w :=
1624   fun (_, hg) => Bool.noConfusion (⟦hg false⟧.symm.trans (readings_world_constant →
1625     hg true))
1626
1627 /-! ## III · RA is keyless and selects nothing -/
1628 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1629
1630 theorem ra_selects_nothing (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → S) :
1631   S :=
1632   h Empty (fun e => nomatch e) (fun e => nomatch e)
1633
1634 /-! ## IV · given one supplied bit, the lock locks exactly one -/
1635
1636 /-- The orientation fibre: the two worlds, swapped by the denial. -/
1637 def deny (w : World) : World := !w
1638
1639 /-- The calibration: with a supplied odd witness `s` and the odd target `d`, exactly
1640 one calibration bit reconciles them on the pair. -/
1641 theorem locks_only_one (s d : World → Bool) (w : World)
1642   (hs : s (deny w) = !s w) (hd : d (deny w) = !d w) :
1643   ∃ c : Bool, (d w = xor (s w) c) ∧ d (deny w) = xor (s (deny w)) c) ∧
1644   ∀ c', (d w = xor (s w) c') ∧ d (deny w) = xor (s (deny w)) c') → c' = c := by
1645   refine (xor (d w) (s w), {?, ?}, {?})
1646   · cases s w <|> cases d w <|> rfl
1647   · rw [hs, hd]; cases s w <|> cases d w <|> rfl
1648   · intro c' (h1, _)
1649     revert h1
1650     cases s w <|> cases d w <|> cases c' <|> intro h1 <|> first | rfl | exact
1651       Bool.noConfusion h1
1652
1653 /-! ## V · the verdict -/
1654
1655 /-- The act: the supplied bit. -/
1656 structure Act where
1657   bit : Bool
1658
1659 def verdict : Option Act → Token
1660 | none => .dot
1661 | some (true) => .sealSep
1662 | some (false) => .sealEq
1663
1664 theorem sealed_at_the_act :
1665   verdict (some (true)) = .sealSep ∧ verdict (some (false)) = .sealEq ∧ verdict
1666   none = .dot :=
1667   (rfl, rfl, rfl)
1668
1669 /-- Both acts are coherent: the lock forms under either supply, so the bit is spent,
1670 never derived. -/
1671 theorem both_acts_lock : verdict (some (true)) ≠ verdict (some (false)) := by decide

```

```

1663 theorem three_seal_lock :
1664   (sealL sentSep = true ∧ sealL sentEq = true) ∧ foldFixed = 0 ∧ lockOf 1 = lockOf 0
1665   (¬ 1) ∧
1666   (¬ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w) ∧
1667   (∀ S : Prop, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → S) → S) ∧
1668   (∀ (s d : World → Bool) (w : World), s (deny w) = ! s w → d (deny w) = ! d w →
1669     ∃ c : Bool, (d w = xor (s w) c ∧ d (deny w) = xor (s (deny w)) c) ∧
1670     ∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c') → c' = c) ∧
1671   (verdict (some (true))) = .sealSep ∧ verdict none = .dot) :=
1672   (tongue_seals_both, form_finds_no_locus, number_reads_one_magnitude,
1673     seals_do_not_point,
1674     ra_selects_nothing, locks_only_one, (rfl, rfl))
1675
1676 end PNP.THREESEAL
1677
1678 /-! ## Cones, pinned: every theorem axiom-free -/
1679 /- info: 'PNP.THREESEAL.tongue_seals_both' does not depend on any axioms -/
1680 #guard_msgs in #print axioms PNP.THREESEAL.tongue_seals_both
1681 /- info: 'PNP.THREESEAL.form_finds_no_locus' does not depend on any axioms -/
1682 #guard_msgs in #print axioms PNP.THREESEAL.form_finds_no_locus
1683 /- info: 'PNP.THREESEAL.number_reads_one_magnitude' does not depend on any axioms -/
1684 #guard_msgs in #print axioms PNP.THREESEAL.number_reads_one_magnitude
1685 /- info: 'PNP.THREESEAL.readings_world_constant' does not depend on any axioms -/
1686 #guard_msgs in #print axioms PNP.THREESEAL.readings_world_constant
1687 /- info: 'PNP.THREESEAL.seals_do_not_point' does not depend on any axioms -/
1688 #guard_msgs in #print axioms PNP.THREESEAL.seals_do_not_point
1689 /- info: 'PNP.THREESEAL.ra_selects_nothing' does not depend on any axioms -/
1690 #guard_msgs in #print axioms PNP.THREESEAL.ra_selects_nothing
1691 /- info: 'PNP.THREESEAL.locks_only_one' does not depend on any axioms -/
1692 #guard_msgs in #print axioms PNP.THREESEAL.locks_only_one
1693 /- info: 'PNP.THREESEAL.sealed_at_the_act' does not depend on any axioms -/
1694 #guard_msgs in #print axioms PNP.THREESEAL.sealed_at_the_act
1695 /- info: 'PNP.THREESEAL.both_acts_lock' does not depend on any axioms -/
1696 #guard_msgs in #print axioms PNP.THREESEAL.both_acts_lock
1697 /- info: 'PNP.THREESEAL.three_seal_lock' does not depend on any axioms -/
1698 #guard_msgs in #print axioms PNP.THREESEAL.three_seal_lock
1699
1700 /-!
1701 # PNP.DEED · the kinetic pulse as the supplied bit
1702
1703 Proposal: supply is the kinetic pulse to examine and compute, and the compute itself
1704 is the supplied bit. This file tests it on the two locks that share the locus.
1705
1706 I · Every deed actuates: a registered compute has positive energy. The deed
1707 instantiates RA at itself. RA's lock is keyless and opens on the deed.
1708 II · A deed's actuation does not certify its output. A compute that prints the
1709 separation and a compute that prints the equality carry the same energy, and
1710 both occur in either world. The pulse is world-constant.
1711 III · The price is symmetric. Registering assent and registering denial each pay
1712 the
1713 floor once. For RA the denial instantiates the axiom it denies; for the
1714 separation the denial instantiates nothing about the separation.
1715 IV · So the deed supplies RA's bit, never the separation's. The compute supplies
1716 its
1717 output, and whether that output is correct is the missing object again: a
1718 procedure whose correctness is proved.
1719
1720 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1721 -/
1722
1723 namespace PNP.DEED
1724
1725 abbrev World := Bool -- true: the separated world; false: the equal world
1726
1727 /- A deed: a registered compute, with its energy and the bit it outputs. -/
1728 structure Deed where
1729   energy : Nat
1730   pos : 0 < energy
1731   output : Bool
1732
1733 /- The output is correct in a world when it matches that world's value. -/
1734 def correct (w : World) (d : Deed) : Prop := d.output = w
1735
1736 /-! ## I · every deed actuates: RA, keyless, opens on the deed -/
1737 theorem deed_instantiates_ra : ∀ d : Deed, 0 < d.energy ⇒ fun d => d.pos
1738
1739 /-! ## II · actuation does not certify the output -/
1740 def assent : Deed := (1, by decide, true)
1741 def denial : Deed := (1, by decide, false)
1742
1743 theorem same_pulse : assent.energy = denial.energy := rfl
1744
1745 /- In every world both deeds occur, and exactly one of them is correct. -/
1746 theorem both_deeds_in_every_world (w : World) :
1747   0 < assent.energy ∧ 0 < denial.energy ∧ (correct w assent ∨ correct w denial) :=
1748   (assent.pos, denial.pos, match w with
1749     | true => 0r.inl rfl
1750     | false => 0r.inr rfl)
1751
1752 /- No reading of the pulse decides correctness: energy does not separate a correct
1753 deed from an incorrect one. -/
1754 theorem pulse_does_not_certify :
1755   ¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output :=
1756   fun (_, hg) => Bool.noConfusion ((hg denial).symm.trans (hg assent))
1757
1758 /- An incorrect deed actuates as fully as a correct one, in every world. -/
1759 theorem wrong_deeds_actuate (w : World) : ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy
1760 :=
1761   match w with
1762   | true => (denial, Bool.noConfusion, denial.pos)
1763   | false => (assent, Bool.noConfusion, assent.pos)
1764
1765 /-! ## III · the price is symmetric -/
1766 def price (floor : Nat) (d : Deed) : Nat := floor * (if d.output then 1 else 1)

```

```

1767 theorem price_symmetric (floor : Nat) : price floor assent = price floor denial :=
1768   rfl
1769
1770 /- RA's denial is itself a deed and instantiates RA; the separation's denial is a
1771 deed and instantiates RA too, which is no information about the separation. -/
1772 theorem denials_both_actuate : 0 < denial.energy ∧ 0 < assent.energy :=
1773   (denial.pos, assent.pos)
1774
1775 /-! ## IV · the deed supplies RA's bit, never the separation's -/
1776 theorem deed_supplies_ra_not_sep :
1777   (∀ d : Deed, 0 < d.energy) ∧
1778   (∀ w : World, ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy) ∧
1779   (¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output) ∧
1780   (∀ floor, price floor assent = price floor denial) :=
1781   (deed_instantiates_ra, wrong_deeds_actuate, pulse_does_not_certify,
1782     price_symmetric)
1783
1784 end PNP.DEED
1785
1786 /-! ## Cones, pinned: every theorem axiom-free -/
1787 /- info: 'PNP.DEED.deed_instantiates_ra' does not depend on any axioms -/
1788 #guard_msgs in #print axioms PNP.DEED.deed_instantiates_ra
1789 /- info: 'PNP.DEED.same_pulse' does not depend on any axioms -/
1790 #guard_msgs in #print axioms PNP.DEED.same_pulse
1791 /- info: 'PNP.DEED.both_deeds_in_every_world' does not depend on any axioms -/
1792 #guard_msgs in #print axioms PNP.DEED.both_deeds_in_every_world
1793 /- info: 'PNP.DEED.pulse_does_not_certify' does not depend on any axioms -/
1794 #guard_msgs in #print axioms PNP.DEED.pulse_does_not_certify
1795 /- info: 'PNP.DEED.wrong_deeds_actuate' does not depend on any axioms -/
1796 #guard_msgs in #print axioms PNP.DEED.wrong_deeds_actuate
1797 /- info: 'PNP.DEED.price_symmetric' does not depend on any axioms -/
1798 #guard_msgs in #print axioms PNP.DEED.price_symmetric
1799 /- info: 'PNP.DEED.denials_both_actuate' does not depend on any axioms -/
1800 #guard_msgs in #print axioms PNP.DEED.denials_both_actuate
1801 /- info: 'PNP.DEED.deed_supplies_ra_not_sep' does not depend on any axioms -/
1802 #guard_msgs in #print axioms PNP.DEED.deed_supplies_ra_not_sep
1803
1804 /-!
1805 # PNP.IDCHECK · is least escape the freedom of prime, or the arrow of RA?
1806
1807 The test is the architecture's own discriminator (Bridge_Final.lean): a keyless
1808 property holds on every frame; a keyed one has a frame inhabiting its denial; no
1809 property is both. Least escape is keyed. Prime freedom, the arrow and RA are
1810 keyless.
1811 So none of them is least escape, on any reading that keeps their content.
1812 Core Lean 4, no import, no axiom of any kind.
1813 -/
1814 namespace PNP.IDCHECK
1815
1816 structure Machine where
1817   Alg : Type
1818   Inst : Type
1819   size : Inst → Nat
1820   out : Alg → Inst → Bool
1821   time : Alg → Inst → Nat
1822   sat : Inst → Bool
1823
1824 def bound (c k n : Nat) : Nat := c * n ^ k + c
1825 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
1826   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
1827 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
1828
1829 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
1830 | 0 => rfl
1831 | n + 1 => beq_self n
1832
1833 def closedWorld : Machine := (Nat, Unit, fun _ => 0, fun _ => false, fun _ => 0,
1834   fun _ => true)
1835 def escapeWorld : Machine := (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A 0, fun _ =>
1836   0, fun _ => true)
1837
1838 theorem closed_closes : NothingEscapes closedWorld :=
1839   fun _ _ => {(), fun (h, _) => Bool.noConfusion h}
1840 theorem escape_fails : ¬ NothingEscapes escapeWorld :=
1841   fun h => match h (0 : Nat) 0 0 with
1842   | {(), hx} => hx (beq_self 0, Nat.zero_le _)
1843
1844 /- Least escape is keyed: true on one coherent machine, false on another. -/
1845 theorem least_escape_is_keyed : NothingEscapes closedWorld ∧ ¬ NothingEscapes
1846   escapeWorld :=
1847   (closed_closes, escape_fails)
1848
1849 /- THE CHECK. No statement that reads the same on every machine is least escape. -/
1850 theorem no_keyless_statement_is_least_escape (P : Prop) :
1851   ¬ ∀ M : Machine, (P ↔ NothingEscapes M) :=
1852   fun h => escape_fails ((h escapeWorld).mp ((h closedWorld).mpr closed_closes))
1853
1854 /-! ## the three candidates, each keyless -/
1855
1856 /- Prime freedom: the fibre of multiplication over 2 is exactly the pair and its
1857 reverse. -/
1858 def primeFreedom2 : Prop := ∀ a b : Nat, a < 3 → b < 3 → (a * b = 2 ↔ (a = 1 ∧ b =
1859   2) ∨ (a = 2 ∧ b = 1))
1860 /- The arrow: an identity exists on every type. -/
1861 def arrowOfRA : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a
1862 /- RA in its formal reading, at any interpretation of its symbols. -/
1863 def raAt (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1864
1865 theorem prime_freedom_is_not_least_escape : ¬ ∀ M : Machine, (primeFreedom2 ↔
1866   NothingEscapes M) :=
1867   no_keyless_statement_is_least_escape _
1868 theorem arrow_is_not_least_escape : ¬ ∀ M : Machine, (arrowOfRA ↔ NothingEscapes M)
1869 :=
1870   no_keyless_statement_is_least_escape _
1871 theorem ra_is_not_least_escape (U : Type) (ΔE : U → Int) :
1872   ¬ ∀ M : Machine, (raAt U ΔE ↔ NothingEscapes M) :=
1873   no_keyless_statement_is_least_escape _

```

```

1861
1862 /-- What does carry least escape: a statement that varies with the machine, as least
1863 escape does. Any such statement that implies it on every machine is keyed in turn.
1864 -/
1865 theorem carrier_must_be_keyed (Q : Machine → Prop) (h : ∀ M, Q M → NothingEscapes M)
1866 :
1867   ¬ Q escapeWorld :=
1868   fun q => escape_fails (h escapeWorld q)
1869
1870 end PNP.IDCHECK
1871
1872 /-- info: 'PNP.IDCHECK.least_escape_is_keyed' does not depend on any axioms -/
1873 #guard_msgs in #print axioms PNP.IDCHECK.least_escape_is_keyed
1874 /-- info: 'PNP.IDCHECK.no_keyless_statement_is_least_escape' does not depend on any
1875 axioms -/
1876 #guard_msgs in #print axioms PNP.IDCHECK.prime_freedom_is_not_least_escape
1877 /-- info: 'PNP.IDCHECK.prime_freedom_is_not_least_escape' does not depend on any
1878 axioms -/
1879 #guard_msgs in #print axioms PNP.IDCHECK.prime_freedom_is_not_least_escape
1880 /-- info: 'PNP.IDCHECK.ra_is_not_least_escape' does not depend on any axioms -/
1881 #guard_msgs in #print axioms PNP.IDCHECK.ra_is_not_least_escape
1882 /-- info: 'PNP.IDCHECK.carrier_must_be_keyed' does not depend on any axioms -/
1883 #guard_msgs in #print axioms PNP.IDCHECK.carrier_must_be_keyed
1884
1885 /--! ## helper cones, pinned -/
1886 /-- info: 'PNP.MISSING.beq_self' does not depend on any axioms -/
1887 #guard_msgs in #print axioms PNP.MISSING.beq_self
1888 /-- info: 'PNP.IDCHECK.beq_self' does not depend on any axioms -/
1889 #guard_msgs in #print axioms PNP.IDCHECK.beq_self
1890 /-- info: 'PNP.IDCHECK.closed_closes' does not depend on any axioms -/
1891 #guard_msgs in #print axioms PNP.IDCHECK.closed_closes
1892 /-- info: 'PNP.IDCHECK.escape_fails' does not depend on any axioms -/
1893 #guard_msgs in #print axioms PNP.IDCHECK.escape_fails

```

Appendix D. The Root Kernel, PNP_Root.lean

The zero-axiom root, sections I, II and V of the TOE edition's kernel TOE_Zero.lean, verbatim: seven theorems, every footprint pinned as *does not depend on any axioms*. SHA-256: 9689dfb47b29697c4276714f9cc70143ab9a55a48b0b27f72d392ed384f7f053

```

1 /--
2   PNP_Root.lean · the root kernel of "The Formal Closure of Computational
3   Separation"
4   The zero-axiom root, sections I, II and V of TOE_Zero.lean, verbatim. A root
5   grounds itself when acts occur and
6   every act instances it. For every such root the root holds, every act re-enacts
7   it, and no external proof adds to
8   it; every denial is an act and re-enacts the root; the root is held by the act
9   itself; no level stands above it;
10  and on the constructed domain, whose one existent actuates at positive energy, the
11  Root Axiom is a theorem. Every
12  law rests on no axiom at all. Core Lean 4, no import, no axiom declared, no sorry,
13  no native_decide.
14 -/
15 namespace PNP.ROOT
16
17 /--! ## I · The self-grounding root: the Omega Boundary -/
18
19 /-- A self-grounding root: acts occur, and every act instances the root. -/
20 structure SelfGrounding (R : Prop) where
21   Act      : Type
22   anAct    : Act
23   instances : Act → R
24
25 /-- A proposition whose denial hands it over. -/
26 def SelfVerifying (P : Prop) : Prop := ¬P → P
27
28 /-- THE OMEGA BOUNDARY: every denial of the root is an act, and re-enacts it. -/
29 theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R
30 := G.instances denial

```

```

24 /-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by the act itself.
25 -/
26 theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R := G.instances
27   G.anAct
28
29 /-- No external proof adds anything to the root. -/
30 theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q : Prop) : (Q
31   → R) → R :=
32   (fun _ => G.instances G.anAct, fun r _ => r)
33
34 /-- The deed of denying the root hands it over. -/
35 theorem denial_instantiates {R : Prop} (G : SelfGrounding R) : SelfVerifying R :=
36   fun _ => G.instances G.anAct
37
38 /-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the root holds,
39   every act re-enacts it, and no
40   external proof adds to it. -/
41 theorem the_floor_is_universal :
42   ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop, (Q → R)
43     → R) :=
44   fun _ G => (seated_undeniable G, denial_reenacts_root G,
45     external_proof_adds_nothing G)
46
47 /--! ## II · The constructed root -/
48
49 /-- The Root Axiom on the constructed one-point domain: its one existent actuates,
50   at positive energy. -/
51 def ΔEₐ : Unit → Int := fun _ => 1
52 def RAₐ : Prop := ∀ x : Unit, 0 < ΔEₐ x
53
54 /-- The constructed domain grounds its root: its one act is a deed, and the deed
55   actuates. -/
56 def raₐ : SelfGrounding RAₐ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)
57
58 /-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
59 theorem root_on_the_constructed_domain : RAₐ := seated_undeniable raₐ
60
61 /--! ## V · The regress terminated -/
62
63 /-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a further ground of the
64   root is idle; the root needs
65   no level above it. -/
66 theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q : Prop, (Q → R) → R :=
67   fun _ _ => G.instances G.anAct
68
69 end PNP.ROOT
70
71 /--! ## The zero law, pinned: every cone is empty -/
72 /-- info: 'PNP.ROOT.denial_reenacts_root' does not depend on any axioms -/
73 #guard_msgs in #print axioms PNP.ROOT.denial_reenacts_root
74 /-- info: 'PNP.ROOT.seated_undeniable' does not depend on any axioms -/
75 #guard_msgs in #print axioms PNP.ROOT.seated_undeniable
76 /-- info: 'PNP.ROOT.external_proof_adds_nothing' does not depend on any axioms -/
77 #guard_msgs in #print axioms PNP.ROOT.external_proof_adds_nothing
78 /-- info: 'PNP.ROOT.denial_instantiates' does not depend on any axioms -/
79 #guard_msgs in #print axioms PNP.ROOT.denial_instantiates
80 /-- info: 'PNP.ROOT.the_floor_is_universal' does not depend on any axioms -/
81 #guard_msgs in #print axioms PNP.ROOT.the_floor_is_universal
82 /-- info: 'PNP.ROOT.root_on_the_constructed_domain' does not depend on any axioms -/
83 #guard_msgs in #print axioms PNP.ROOT.root_on_the_constructed_domain
84 /-- info: 'PNP.ROOT.no_level_above' does not depend on any axioms -/
85 #guard_msgs in #print axioms PNP.ROOT.no_level_above

```

Author's Provenance and Method Disclosure

The arc was found within Trisduction, the author's verification architecture, through its operating system PhysOS^T (Islam 2026c), and compiled with the assistance of an AI scribe; every theorem stands on its compiled proof, reproducible by any reader from the appendices. The author declares no competing interests. Correspondence: islam@alumni.iu.edu.